



AI Services Strategic Partnership WS2 Engineering Modernization

Technical Design Document

Authors: Google PSO Team

Shubha Mohan , Sean Rastatter , Navneet Kamboj , Nirmal Thumbar , VINITA MISHRA (xWF) ,
Kolli Pratheek (xWF) , dimosthenis@google.com

July 2025

Contents

Executive Summary	6
1. Introduction	8
2. Use Case Description	8
3. Infrastructure set-up	8
4. Solution Overview	19
4.1. Ingestion Pipeline	19
4.1.1. Purpose	19
4.1.2 Architecture	20
4.2. Text Extraction (Q&A RAG)	21
4.2.1. Purpose	21
4.2.2. Architecture	22
4.3. Document Generation	23
4.3.1. Purpose	23
4.3.2. Architecture	23
4.4. Image Digitization	24
4.4.1 Purpose	25
4.4.2 Architecture	25
4.5. Deliverables	26
Ingestion Pipeline Deliverables:	26
Text Extraction (Q&A RAG) Deliverables:	26
Document Generation Deliverables:f	27
Image Digitization Deliverables:	27
5. Ingestion Pipeline	28
5.1. Introduction	28
5.2. Architecture	28
5.3. End-to-End Flow	32
5.4. Key Design Decisions	34
6. Text Extraction	35
6.1 Introduction	35
6.2 Background	35
6.3 Architecture	40
6.4 End-to-End Flow	43
6.5 Key Design Decisions	45
7. Document Generation	46
7.1. Introduction	46
7.2. Architecture	47
7.3. End-to-End Flow (Detailed Analysis)	49
7.4. Key Design Decisions	51

8. P&ID Image Recognition	52
8.1 Architecture	53
8.2 End-to-End Flow	55
8.3 Key Design Decisions	59
9. Limitations	60
9.1 Token limit	60
9.2. End-to-End Processing Time	60
9.3. Quality of Dataset and Model Accuracy	61
9.4. Quantity and Quality of Metadata	61
9.5. User Interface and Discoverability	61
9.6. Difficulty in Handling Ambiguous Queries	62
10. Observability	62
10.1 Cloud Logs	62
Ingestion Pipeline	63
Text Extraction (Q&A)	63
Document Generation	63
Image Digitization	64
10.2 Cloud Run Jobs	64
10.3 Vertex AI Endpoints	64
11. Performance Testing	65
11.1 Testing Methodology and Tools	65
11.2 Key Findings and Improvements	65
11.3 Detailed Test Results	66
11.4 Conclusion and Further Considerations	67
12. Error Handling	69
12.1. Ingestion Pipeline Error Handling	69
12.2. Text Extraction (Q&A) Error Handling	70
12.3. Document Generation Error Handling	70
12.4. Image Digitization Error Handling	71
13. Recommendations and Best Practices	72
13.1 Text Extraction	72
13.1.1. Recommended Modifications for Citations Improvement	72
13.1.2. Implementing Hybrid Search	72
13.2 Document Generation and Ingestion pipeline	73
13.3 Agent	74
13.4 P&ID Image Extraction	77
13.4.1 Data Augmentation	77
13.4.2 Human-in-the-loop validation and iterative model improvement	78
14. Code Repository	78
14.1. Repository Details	78

14.2. Branching Strategy	78
14.3. Directory Structure	78
15. Honeywell Security Requirements	81
15.1 Data Encryption	81
15.2 Secure Network Infrastructure	81
15.3 Access Control:(Authentication Authorization)	81
15.4 Firewalls and Intrusion Prevention Systems:	81
15.5 Anti-Malware and Anti-Virus Solutions/ DDoS Protection:	82
15.6 API Security:	82
15.7 Logging and Incident Response:	82
15.8 Data Privacy:	82
15.9 Secure Development:	82
15.10 Regular Updates and Patch Management:	82
15.11 Data Backup and Recovery:	83
15.12 User Training and Awareness:	83
15.13 Data Retention Policies:	83
15.14 Secure Deployment Environment:	83
15.15 Regular Security Testing:	83
15.16 Secure User Interfaces:	83
15.17 Documentation and Training Materials:	84
15.18 Vendor Security Assessment:	84
16. Future Scalability Considerations	85
17. Glossary of Acronyms	87

Executive Summary

The Google Cloud Professional Services Organization (PSO) worked on a Knowledge Assist Project to drive Honeywell's Engineering Modernization initiative. The project directly addresses critical industry challenges posed by impending labor and skills shortages, and the vast, previously inaccessible knowledge embedded within Honeywell's historical documents and images.

The solution leverages advanced AI capabilities to deliver three primary functionalities, integrated into one agent:

1. **Knowledge Assist:** A multi-turn chatbot, built using Google's Gemini LLM and Langchain, providing rapid, intuitive access to information from hundreds of thousands of engineering and product documents.
2. **Image Digitization:** A sophisticated system to transform Piping and Instrumentation Diagrams (P&IDs) into structured JSON formats by intelligently identifying symbols, lines, and connections, enabling their use in downstream HMI systems.
3. **Document Generation:** An automated capability to create professional proposals and design documents by synthesizing content from customer inputs, content libraries, and historical data.

The underlying architecture is highly scalable and robust, built on Google Cloud Platform (GCP) using serverless compute (Cloud Run, Cloud Functions), Vertex AI for advanced machine learning models and vector search, Document AI for intelligent document processing, and Firestore for rich metadata management. It incorporates a comprehensive Ingestion Pipeline for continuous data updates, a Retrieval-Augmented Generation (RAG) framework for context-aware content, and robust error handling and observability mechanisms. The design also adheres strictly to Honeywell's security requirements, including data encryption, secure networking, and stringent access controls.

Personas:

- Knowledge Assist and Image digitization is primarily meant for use by the engineers, based off of what has been ingested into the system thus far. It will eventually include a broader spectrum of users from executives down to technicians.
- Document Generation: Generation of proposals is being leveraged by P&E group while the FDS generation is primarily used by engineers. Future enhancements to the kind of documents generated will mean this solution can be leveraged by a broader set of users.

This modernization effort is designed to significantly optimize Honeywell's industrial processes, empower its workforce by unlocking buried knowledge, and drive meaningful business outcomes through enhanced efficiency and accuracy.

1. Introduction

This Technical Design Document serves as technical documentation of the work done by the Google Cloud Professional Services Organization (PSO) team to support Engineering modernization at Honeywell. In the sections to follow, we will describe the Generative AI solutions built to support 1) Knowledge Assist, a chatbot for accessing information from 100s of 1000s of documents 2) Image Digitization of P&ID diagrams and 3) Generation of proposal and design documents.

We will finalize the document defining a set of recommendations on next steps and future scaling considerations.

2. Use Case Description

With the mass retirement of workers from the baby boomer generation, the industrial sector faces both labor and skills shortages, and AI should be part of the solution – as a revenue generator, not job eliminator. A lot of knowledge is buried in Honeywell’s historical documents and images and this project brings the transformative power of AI to industrial operations. This project has enabled Honeywell to optimize processes, empower the workforce and drive meaningful business outcomes , through the 3 AI agents below.

The “Knowledge Assist” agent, built using Gemini and Langchain, currently leverages Maintenance Assist as the front-end and surfaces information from 100s of 1000s of engineering and product documents through an intuitive, multi-turn chatbot interface.

The “Image Digitization” agent brings together multiple models to identify symbols, lines, thickness of lines and connections to convert P&ID diagrams into JSONs that can be used for downstream processing by the HMI machine.

The “Document Generation” agent brings together content from customer input documents such as RFPs, content libraries and historical information to generate Functional Design Specifications and Technical proposals.

3. Infrastructure set-up

All recommended infrastructure setup can be found at [this spreadsheet](#). We have listed below the setup as well.

Adhering to best practices, the key principles followed are:

1. All work at each stage will be performed only by service accounts
2. Each component of the extraction pipeline will use the same service account
3. That service account should have the following permissions

3.1 APIs

API Name	API URL
Vertex AI API	aiplatform.googleapis.com
Artifact Registry API	artifactregistry.googleapis.com
BigQuery API	bigquery.googleapis.com
Cloud Functions API	cloudfunctions.googleapis.com
Cloud Resource Manager API	cloudresourcemanager.googleapis.com
Cloud Scheduler API	cloudscheduler.googleapis.com
Compute Engine API	compute.googleapis.com
Identity Access Management (IAM) API	iam.googleapis.com
IAM Service Account Credentials API	iamcredentials.googleapis.com
Cloud Pub/Sub API	pubsub.googleapis.com
Cloud Run Admin API	run.googleapis.com
Cloud Storage API	storage.googleapis.com
Discovery Engine API	discoveryengine.googleapis.com
Cloud Build API	cloudbuild.googleapis.com
Cloud Document AI API	documentai.googleapis.com
Cloud Workflows API	workflows.googleapis.com
Cloud Firestore API	firestore.googleapis.com
Cloud Memorystore for Redis API	redis.googleapis.com
Cloud Monitoring API	monitoring.googleapis.com
Cloud Logging API	logging.googleapis.com
Notebooks API	notebooks.googleapis.com

3.2 Recommended Infrastructure Setup

Associated architecture diagrams can be found at [this link](#).

IAM			
IAM > Service Accounts			

Resource	Description	IAM Roles	Notes
document-ingestion-workflows-sa@{project_id}.gserviceaccount.com	Service Account to be used to run data ingestion workflows in GCP Workflows.	Cloud Functions Admin Cloud Functions Service Agent Cloud Run Admin Eventarc Event Receiver Pub/Sub Publisher Service Account Token Creator Service Account User Storage Admin Storage Object Admin Workflows Admin	Terraform
spyder-agent-api-sa@{project_id}.gserviceaccount.com	Service account used for the various services (Cloud Functions and Cloud Runs) related to the Spyder Agent.	BigQuery Data Editor BigQuery Job User Cloud Functions Admin Cloud Functions Service Agent Cloud Run Admin Discovery Engine Admin Document AI Administrator Firebase Admin Logs Writer Monitoring Editor Pub/Sub Publisher Secret Manager Secret Accessor Service Account Admin Service Account Token Creator Service Account User Service Usage Admin Storage Admin Storage Object Admin Vertex AI Administrator Workflows Admin	Terraform
data-science-jobs-sa@{project_id}.gserviceaccount.com	Service account to be used for data science development and experimentation. Will be primarily used within Vertex AI Workbench instances.	Artifact Registry Writer BigQuery Admin BigQuery Data Editor BigQuery Job User Cloud Build Editor Cloud Functions Admin Cloud Run Admin Dataproc Editor Document AI Administrator Pub/Sub Publisher Service Account Admin Service Account Token Creator Service Account User Service Usage Admin Storage Admin Storage Object Admin	Terraform

		Vertex AI Administrator Workflows Admin	
Storage			
Storage > Buckets			
Resource	Description	Notes	
gs://{env}-spyder-etl-staging	Used as a staging location for data ingestion jobs into Vector Search Indices. Convertex file types and split files are placed in this bucket, as well as a list of all failed and succeeded ingestion jobs.	Terraform	
gs://{env}-spyder-etl-vectorstore-index	Used by Vector Search as an intermediate location during data ingestion. Doc AI Parser outputs are dumped here as json files.	Terraform	
gs://{env}-projectassist	Used to store mvp documents moved over from Sharepoint.	Terraform	
Networking			
Networking > VPC Networks			
Resource	Description	Notes	
{env}sharedvpc (project: {project_id})	Shared VPC across the different workstreams.	Terraform	
Networking > VPC Subnets			
Resource	Description	Notes	
{project_id}-{region}-{env}	Subnetwork for project assist.	Terraform	
Networking > Serverless VPC Access Connector			
Resource	Description	Notes	
{env}-project-assist-connector (under region: {region}, vpc:	This VPC connector is used to allow our services to access	Terraform	

{env}sharedvpc, and subnet: corp-entit-projectassist-{region}-{env)})	certain internal honeywell servers (e.g. smtp-secure.honeywell.com)		
Databases			
Databases > Firestore Instances			
Resource	Description	Notes	
document-metadata	Firestore database used for storing metadata about ingested documents. Metadata was originally produced by analyzing the first 2 pages of documents. Metadata is now coming from business teams.	Terraform	
Serverless			
Serverless > Cloud Functions			
Resource	Description	Notes	
spyder-etl-write-processing-status	Takes in a list of files (successfully ingested files and failed ingested files) and writes them out GCS as text files	gcloud or Terraform; uses document-ingestion-workflows-sa@{ project_id}.gserviceaccount.com.	
spyder-etl-split-large-files	Takes in a list of files (successfully ingested files and failed ingested files) and writes them out GCS as text files	gcloud or Terraform; uses document-ingestion-workflows-sa@{ project_id}.gserviceaccount.com.	
spyder-etl-read-files-to-process	Looks at a specified bucket location. Within this bucket exists files, each containing a list of 500 documents to be ingested. This function looks at the oldest file in the bucket, reads this list of 500 files, deletes the oldest file and then sends along this list of 500 files for further process. If	gcloud or Terraform; uses document-ingestion-workflows-sa@{ project_id}.gserviceaccount.com.	

	no files exists, this function will return nothing.		
spyder-etl-document-ingestion	Performs the full scale document ingestion process. This will batch parse files (in a parallel manner) using Doc AI, and then embed doc chunks and store them in vector search.	gcloud or Terraform; uses document-ingestion-workflows-sa@{project_id}.gserviceaccount.com.	
spyder-agent-document-generation-handler	This function handles document generation tasks by going through specific document templates and filling in the <prompt> tags content.	gcloud or Terraform; uses spyder-agent-api-sa@{project_id}.gserviceaccount.com; Attached to VPC network with the following details ({env}-project-assist-connector (under region: {region}, vpc: {env}sharedvpc, and subnet: corp-entit-projectassist-{region}-{env})))	
spyder-etl-check-new-files	Looks at a given bucket and grabs a list of files that were written to this bucket within the past 5 minutes. This function is intended to be trigger by a cloud scheduler, which allows us to run batch ingestion jobs progressively over time.	gcloud or Terraform; uses document-ingestion-workflows-sa@{project_id}.gserviceaccount.com.	
Serverless > Cloud Run			
Resource	Description	Notes	
spyder-agent-token-exchange-svc	This service takes in a base 64 encoded auth header, pulls out the session id, verifies the session is active using maintainence assist's auth API, generates a valid GCP token, reformats the request/response to/from the backend Spyder agent and then forwards along the request.	gcloud or Terraform; uses spyder-agent-api-sa@{project_id}.gserviceaccount.com	
spyder-agent-runtime-env	This service runs a flask api which provides an endpoint for interacting with the spyder agent.	gcloud or Terraform; uses spyder-agent-api-sa@{project_id}.gserviceaccount.com	

Serverless > Cloud Run Jobs			
Resource	Description	Notes	
spyder-etl-docx-to-pdf-job	This job takes in a list of GCS URIs of files that are non-Pdf files and converts them to Pdfs, then writes them to a specified bucket.	gcloud or Terraform; uses document-ingestion-workflows-sa@{project_id}.gserviceaccount.com.	
CI/CD			
CI/CD > Artifact Registry Repositories			
Resource	Description	Notes	
spyder-agent-images	Artifact repository for the Cloud Run (container) services	Terraform	
Integration Services			
Integration Services > Cloud Schedulers			
Resource	Description	Notes	
spyder-etl-scheduler	Scheduler that invokes a Cloud Workflow on a recurring time frame. Currently paused and set to run every 40 minutes. This scheduled time changes based on the needs of the ingestion job.	Terraform	
Integration Services > Cloud Workflows			
Resource	Description	Notes	
spyder-etl-incoming-doc-checker	Workflow that calls the spyder-check-new-files-v1 cloud function and returns a list of files that have been added in the past few minutes	gcloud or Terraform; uses document-ingestion-workflows-sa@{project_id}.gserviceaccount.com.	

	(specified about of time).		
spyder-etl-document-ingestion-wf-v1	Workflow that runs the full document ingestion pipeline, which calls the following services/jobs: spyder-read-files-to-process -> docx-to-pdf-job -> spyder-split-large-files -> spyder-document-ingestion -> spyder-write-processing-status	gcloud or Terraform; uses document-ingestion-workflows-sa@{project_id}.gserviceaccount.com.	
spyder-agent-document-generation-workflow	Workflow that gets triggered by the document generation tool inside of the backend agent. This workflow will async run a doc generation task by calling the spyder-document-generation-handler-v2 service.	gcloud or Terraform; uses document-ingestion-workflows-sa@{project_id}.gserviceaccount.com.	
Artificial Intelligence			
Artificial Intelligence > Vertex AI Workbench Instances			
Resource	Description	Notes	
data-science-development	Workbench instance used for development.	Terraform. It is attached to the following subnetwork: {project_id}-{region}-{env}.	
Artificial Intelligence > Document AI Parsers			
Resource	Description	Notes	
spyder-etl-ocr-parser	Document OCR parser	Terraform	only processor can be created via terraform
Artificial Intelligence > Vector Search Indices			
Resource	Description	Notes	

spyder_agent_index	Index for the Vector Search Index.	Terraform	https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/vertex_ai_index
Artificial Intelligence > Vector Search Endpoints			
Resource	Description	Notes	
spyder_agent_endpoint	Endpoint for the Vector Search Index.	Terraform	

3.3 GCP Models

Model Name	Region	Notes
gemini-1.5-pro-002	us-central1	Standard model being used for the agent engine.
gemini-2.0-flash	us-central1	Model that was tested; faster response time, marginally different performance quality.
text-embedding-004	us-central1	Model used for embeddings. We believe there will be significant improvement if we are allowed to use text-embedding-005.

* It is recommended that the models be upgraded to gemini-2.5-pro and text-embedding-005.

The current implementation across the system explicitly uses the **text-embedding-004** model. This model was selected as it was the latest, most stable version available at the time of initial development, offering a strong balance of performance, cost, and a 768-dimensional output suitable for our vector search indexes.

An upgrade to a newer model like **text-embedding-005** is a natural next step for future enhancement. The anticipated advantages would be:

- **Improved Semantic Representation:** Newer models are generally trained on more data and may offer more nuanced understanding of technical and domain-specific language, potentially improving retrieval relevance.
- **Performance/Cost Efficiency:** Newer models can sometimes offer better performance at a lower cost per token.

Upgrade to gemini-2.5-pro is recommended as it outperforms gemini-1.5-pro across all benchmarks.

You can find more information [here](#).

3.4 Quota Increases

Request number	Type	Service	Quota	Dimensions (e.g. location)	Original value	Requested value
1be1b051864749648a	User	Vertex AI API	Regional online prediction requests per base model per minute per region per base_model	region : us-central1 base_model : textembedding-gecko	1,500	5,000
56991116	User	Vertex AI API	Query Reasoning Engine requests per minute per region	region : us-central1	60	200
56991102	User	Cloud Document AI API	Number of online process document requests (US) per minute per processor_type	processor_type : OCR_PROCESSOR	120	500
56993090	User	Cloud Document AI API	Number of concurrent batch prediction requests processed using document processor (US)		5	20
56991102	User	Cloud Document AI API	Number of API requests per minute		1,800	5,000

- **Cloud Document AI - Concurrent Batch Requests:**

- **Quota Description:** "Number of concurrent batch prediction requests processed using document processor (US)". The request was to increase this from **5 to 20**.
- **System Impact:** This is arguably the **most critical quota for our Ingestion Pipeline**. The `spyder-doc-ingestion-wf-v1.yaml` is designed to process documents in parallel, with a concurrency limit set in the workflow. However, the actual parallelism for the parsing step is ultimately constrained by this Document AI quota. With a limit of 5, only 5 documents can be parsed by the service simultaneously, regardless of how many parallel threads our workflow spawns. This creates a bottleneck that serializes the processing. Increasing this to 20 directly enables higher throughput, significantly reducing the end-to-end time required to ingest a large batch of documents.
- **Vertex AI API - Reasoning Engine / LLM Requests:**
 - **Quota Description:** "Query Reasoning Engine requests per minute per region". The request was to increase this from **60 to 200**.
 - **System Impact:** This quota directly affects the responsiveness and capacity of our user-facing features: **Text Extraction (Q&A)** and **Document Generation**. A limit of 60 requests per minute means the system can only handle, on average, one user query per second. This is insufficient for supporting multiple concurrent users. For the Document Generation feature, which makes numerous parallel RAG calls to the LLM, this low limit would serialize the content generation and dramatically increase the time to create a document. The requested increase to 200 RPM is essential for providing a responsive user experience and enabling efficient document generation.
- **Vertex AI API - Embedding Requests:**
 - **Quota Description:** "Regional online prediction requests per minute per base model" for `textembedding-gecko` (the family for our `text-embedding-004` model). The request was to increase this from **1,500 to 5,000**.
 - **System Impact:** This quota impacts the **Ingestion Pipeline**. After Document AI splits a document into chunks, each chunk must be sent to the embedding model. A document can have hundreds of chunks. A higher quota allows the `spyder-document-ingestion` function to generate embeddings for all chunks of a large document, or for multiple documents in parallel, much more quickly, thus speeding up the indexing stage of the ingestion process.
- **Cloud Document AI - General & Online API Requests:**
 - **Quota Description:** "Number of online process document requests" and "Number of API requests per minute".
 - **System Impact:** While our primary ingestion pipeline uses the batch processing API, these quotas are important for overall system health and future flexibility. The general "Number of API requests per minute" provides headroom for all interactions with the Document AI service. The "online process" quota, while not used by the main batch workflow, would be critical if we were to implement real-time, single-document

ingestion in the future. Securing these increases ensures the service remains responsive and can accommodate future architectural changes.

3.5 Infrastructure-as-Code

All associated terraform modules can be found under the *terraform* branch:

<https://github.com/HON-CORP/Project-SPYDER-WS2-GCP/tree/terraform/terraform/honeywell-terraform>

These modules correspond to the documented infrastructure setup and configuration as explained above in section 3.

4. Solution Overview

This section provides a comprehensive overview of the Spyder Project Assist system, detailing its core functional components: **Ingestion Pipeline**, **Text Extraction (Q&A RAG)**, and **Document Generation**. Each component is designed to address specific needs, from processing raw data to delivering intelligent, context-aware responses and artifacts to the user. The overall architecture aims for scalability, resilience, and maintainability by leveraging Google Cloud's serverless and managed services.

4.1. Ingestion Pipeline

4.1.1. Purpose

The **primary purpose** of the Ingestion Pipeline is to **automate the ingestion and transformation of raw, unstructured documents from diverse sources into a structured, queryable knowledge base (Vector DB)**. This is fundamental for the effectiveness of the downstream RAG (Retrieval-Augmented Generation) capabilities.

Specific goals include:

- **Data Standardization:** Converting various input file formats (e.g., DOCX, PPTX, XLSX) into a uniform PDF format suitable for consistent processing.
- **Content Extraction & Chunking:** Intelligently extracting textual content and breaking it down into semantically meaningful chunks that preserve context, using advanced layout-aware models.

- **Metadata Enrichment:** Associating each document and its chunks with rich metadata (e.g., project number, GBE, document type) fetched from Firestore, which is crucial for filtered retrieval.
- **Vectorization and Indexing:** Generating vector embeddings for text chunks and indexing them in a specialized vector database for efficient semantic search.
- **Continuous Updates & Scalability:** Ensuring that the knowledge base is continuously updated with new and modified documents, and that the pipeline can scale to handle large volumes of data.
- **Robustness and Monitoring:** Providing detailed logging and status reporting for each stage of the ingestion process to ensure reliability and facilitate troubleshooting.

4.1.2 Architecture

The Ingestion Pipeline is a **serverless, event-driven system orchestrated by Cloud Workflows**.

Key components and their interactions:

- **Cloud Schedulers** initiate the process by triggering the `spyder-incoming-doc-checker.yaml` (or `...-doc-gen.yaml`) workflow.
- This initial workflow invokes the `spyder-check-new-files-v1` **Cloud Function**. This function scans designated GCS buckets for new files based on `blob.time_created`, intelligently filtering out URIs from document generation output folders (like `/document_generation/.../templates/` or `/document_generation/.../output/`) to prevent re-ingestion loops. It then batches the URIs of new files into `.txt` chunk files in GCS.
- The main `spyder-doc-ingestion-wf-v1.yaml` **Cloud Workflow** takes over:
 - It calls the `spyder-read-files-to-process` **Cloud Function** to retrieve a batch of URIs from a GCS chunk file.
 - In parallel, it processes each URI:
 - Microsoft Office documents are converted to PDF by the `docx-to-pdf-job-v3` **Cloud Run job**, which utilizes a headless LibreOffice instance.
 - The resulting PDFs (or original PDFs) are then processed by the `spyder-split-large-files` **Cloud Function**. This function splits PDFs larger than 15MB, crucially prepending the first two pages of the original document to each subsequent chunk to maintain context.
 - The final set of PDF URIs is passed to the `spyder-document-ingestion` **Cloud Function**. This is where:

- Metadata is fetched from **Firestore** using `get_metadata_for_filename`.
- **Document AI** (specifically the Layout Parser Processor, as shown in the "RAG Data Ingestion Process" diagram) is used for semantic chunking (`DocAIParser` with `include_ancestor_headings=True`).
- Embeddings are generated for these chunks using a **Vertex AI Embeddings Model**.
- The chunks, their embeddings, and metadata are upserted into **Vertex AI Vector Search** (the "Vector Search Index" in the diagram).
- Finally, the workflow calls the `spyder-write-processing-status` **Cloud Function** to log detailed success/failure information to GCS and send an email summary.

4.2. Text Extraction (Q&A RAG)

4.2.1. Purpose

The **primary purpose** of the Text Extraction (Q&A RAG) capability is to **provide users with accurate, concise, and contextually relevant answers to their natural language queries**. This involves deeply understanding the query, retrieving pertinent information from the indexed knowledge base, and synthesizing an answer grounded in that information.

Specific goals include:

- **Natural Language Understanding:** Effectively parsing and interpreting user queries, including follow-up questions that rely on conversational history.
- **Targeted Information Retrieval:** Identifying and fetching the most relevant document chunks from the Vector Search index, using both semantic similarity and metadata-based filtering.
- **Contextual Grounding:** Ensuring that the generated answers are based solely on the retrieved information to prevent hallucination and provide factual responses.
- **Source Citations:** Providing users with references to the source documents from which the answer was derived, enabling verification and further exploration.
- **Conversational Capability:** Maintaining context across multiple turns of a conversation to answer follow-up questions effectively.

4.2.2. Architecture

The Text Extraction flow is orchestrated by the **Spyder Agent**, hosted within the **Agent Runtime (Cloud Run / Flask)**.

Key components and their interactions:

1. **Agent Runtime** (`agent_runtime/main.py`):
 - Receives the user's query via its `/query` Flask endpoint.
 - Calls `get_search_filter` (from `utils.py`) to parse the query, extract entities (project numbers, project/customer names using LLMs and `rapidfuzz`), and validate them against **Firestore**. This creates a metadata filter (e.g., `{"project_num": ["USP-019208"], "GBE": ["tenant_id"]}`).
2. **Langchain React Agent** (`agent_runtime/agent.py`):
 - The `SpyderAgentManager` initializes a ReAct-based agent using `create_react_agent`. This agent is provided with a `PromptTemplate` instructing it on tool usage and the `document_search` tool.
 - The agent receives the user's query and the generated filter as a delimited string and determines that `document_search` is the appropriate tool.
3. **document_search Tool** (`agent_runtime/tools.py`):
 - This tool orchestrates the RAG process by calling `create_multi_pdf_engine`.
 - `create_multi_pdf_engine` **function**:
 - Sets up the **Vertex AI Embeddings Model** and **Vertex AI ReRanker**.
 - Connects to **Vertex AI Vector Search** (`VectorSearchVectorStore`) using the `VS_INDEX_IDS` and applies the `formatted_search_filter`.
 - The retriever uses **Contextual Compression** with the re-ranker to refine search results.
 - A **history-aware retriever** (`create_history_aware_retriever`) reformulates the user's question to be standalone if it depends on `chat_history`.
 - An **answer generation chain** (`create_stuff_documents_chain`) is built using a **Gemini 1.5 Pro LLM**. Its system prompt (`qa_system_prompt`) explicitly instructs the LLM to answer based *only* on the provided context ("Document Chunks" from the diagram) and to state "I don't know" if the answer is not found.
 - The `rag_chain.invoke()` call executes this entire RAG sequence: Embed User Query -> Filter w/ Metadata -> Nearest Neighbor Search -> ReRank -> Generate Answer.
 - Finally, it calls `retrieve_custom_metadata_from_firestore` to fetch details for the cited source documents.
4. **Response Formulation**: The `document_search` tool returns the generated answer and source citations to the agent, which then passes it back through the Agent Runtime to the user.

4.3. Document Generation

4.3.1. Purpose

The **primary purpose** of the Document Generation feature is to **automate the creation of structured, standardized documents by dynamically populating pre-defined templates with relevant, AI-generated content**. This significantly reduces manual effort, ensures consistency, and leverages the system's RAG capabilities to produce high-quality, data-driven artifacts.

Specific goals include:

- **Template-Driven Automation:** Allowing users to define document structures and boilerplate text in standard `.docx` templates.
- **Intelligent Content Population:** Using the powerful RAG engine to generate specific content for designated sections within the templates.
- **Contextual Relevance:** Ensuring that the generated content is tailored to the specific context provided by the user (e.g., for a particular project or customer).
- **Formatting Preservation and Enhancement:** Maintaining the original template's formatting and applying rich text formatting (lists, bolding, etc.) to the AI-generated content.
- **Asynchronous Delivery:** Providing a responsive user experience by queuing the generation task and delivering the final document via email.

4.3.2. Architecture

The Document Generation process, while initiated by the **Spyder Agent**, quickly transitions to an asynchronous backend workflow, as illustrated in the interactions between the "Spyder Agent" (selecting the "Text Doc Gen" tool) and the subsequent workflow leading to the generation handler.

(Focus on "Spyder Agent" -> "Text Doc Gen" tool interaction, and the implied offloading to a backend process)

Key components and their interactions:

1. **Agent Runtime (`agent_runtime/main.py` & `agent.py`):**
 - Similar to Text Extraction, the user query is received, and `get_search_filter` is called to extract entities and form filters. A **project/opportunity number is mandatory** for this flow.
 - The Langchain React Agent identifies the intent to generate a document and selects the `document_generation` tool.

2. `document_generation Tool` (`agent_runtime/tools.py`):
 - This tool is responsible for initiating the asynchronous generation process.
 - It calls `select_template_controlled` (from `utils.py`) which uses an LLM and `rapidfuzz` to robustly match the user's request to a specific `.docx` template stored in **GCS** (`GCS_TEMPLATE_FOLDER`).
 - It retrieves/validates the `Project Name` and `GBE (Tenant ID)`, fetching from **Firestore** if necessary using `get_gbe_from_firestore` and `retrieve_project_name_from_firestore`.
 - It then constructs an `args` payload and triggers the `spyder-doc-gen-workflow.yaml` **Cloud Workflow** via `client.create_execution`.
3. **Cloud Workflow** (`spyder-doc-gen-workflow.yaml`):
 - This workflow acts as a simple, durable message passer. It receives the `args` payload and makes an authenticated HTTP POST request (using `OIDC`) to the `DOC_GEN_URL`, which points to the `spyder-document-generation-handler`.
4. **Document Generation Handler**
(`spyder-document-generation-handler/utils/document_handler.py`):
 - The `document_generation_task_handler` function is invoked.
 - It loads the specified `.docx` template from GCS using `load_docx_from_gcs`.
 - It parses the document, identifying sections marked by `<prompt>...</prompt>` tags and replacing them with unique `<placeholder_{uuid}>` IDs.
 - For each unique prompt, it **concurrently** calls the `document_search_prioritized` RAG engine (the same one used in Text Extraction, referred to as the "Rational Chain" in the diagram) to generate content. This involves querying **Vector Search** and using an LLM.
 - The generated content (which can include markdown) is then injected back into the document, replacing the placeholders. The `add_markdown_to_paragraph` utility converts markdown to rich DOCX formatting.
 - The final document is uploaded to a structured path in **GCS** and then emailed to the user.

4.4. Image Digitization

4.4.1 Purpose

The Image Digitization process aims to **transform PID images (Piping and Instrumentation Diagrams) into a digital format**. This conversion significantly diminishes the manual work and time needed to integrate these images into the HMI systems utilized by

Honeywell. By streamlining this process, Honeywell can efficiently develop dashboards for these PIDs.

Specific goals include:

- **Image Analysis:** Identify and classify symbols, vessels, and specialized line types within Piping and Instrumentation Diagram (PID) images.
- **Line Recognition:** Differentiate and detect both continuous and segmented lines present in the PID images.
- **Information Retrieval:** Extract textual data from the images and accurately link it with the corresponding graphical symbols.
- **Network Development:** Create an interconnected graph structure where individual symbols are represented as nodes, and their relationships are denoted by edges.

4.4.2 Architecture

The image digitization process operates as an event-driven pipeline controlled by Cloud workflows.

Key components and their interactions:

1. **Cloud Workflow:**
 - Oversees the end-to-end image digitization process.
 - Initiates image extraction when files are uploaded to the designated Google Cloud Storage (GCS) bucket.
2. **Cloud Run Job:**
 - Carries out essential digitization tasks.
 - Includes symbol, vessel, and line type identification and categorization, line recognition, and graph generation.
 - Saves the output to the target GCS bucket.
3. **Document AI:**
 - Enables text extraction from Process and Instrumentation Diagram (PID) images through the Document Optical Character Recognition (OCR) processor.
4. **VertexAI:**
 - **VertexAI Datasets:** Functions as the storage location for different versions of datasets used to train object detection models.
 - **VertexAI Training Pipeline:** Utilizes VertexAI training pipelines and Automated Machine Learning (AutoML) for training object detection models.
 - **VertexAI Endpoint:** Hosts the deployment of trained models for the detection of symbols, vessels, and specific line types.
 - **VertexAI Batch Prediction:** Facilitates vessel detection in images exceeding 1.5 MB.

4.5. Deliverables

The Spyder Project Assist system, through its integrated components, will deliver the following key outcomes and artifacts:

Ingestion Pipeline Deliverables:

- **Continuously Updated Vector Knowledge Base:** A robust and scalable index in **Vertex AI Vector Search** containing embedded document chunks, enriched with filterable metadata.
- **Standardized Document Archive:** Original and processed (PDF format) documents stored systematically in **Google Cloud Storage**.
- **Comprehensive Processing Logs:** Detailed logs in GCS for each ingestion run, cataloging successfully processed files, failures, and specific error messages for easier troubleshooting and monitoring.
- **Enriched Metadata Repository:** An up-to-date **Firestore** collection (`qa-document-metadata`) linking documents to their critical business attributes (Project Number, GBE, Customer Name, Document Type, etc.).

Text Extraction (Q&A RAG) Deliverables:

- **Accurate, Context-Grounded Answers:** Users receive precise answers to their natural language queries, derived directly from the ingested document corpus.
- **Source Citations:** Answers are accompanied by citations (including metadata like filename and URL fetched from Firestore) to the source documents, allowing for transparency, verification, and deeper exploration by the user.

How Citations Are Derived: The citation mechanism is directly tied to the RAG process in the `document_search` tool.

1. The `rag_chain.invoke()` call returns a final answer, but it also returns the list of `Document` objects that were passed into the LLM as context.
2. Each of these `Document` objects contains a `metadata` attribute, which includes a `source` field holding the GCS URI of the chunk's original document.
3. The code extracts these GCS URIs from the top-ranked source documents.
4. It then calls the `retrieve_custom_metadata_from_firestore` utility, which uses the filename from the GCS URI to look up richer, more user-friendly metadata (like the original SharePoint URL or document title) from our Firestore database. This enriched metadata is what is presented to the user as the citation.

- **Conversational Interaction:** The ability for users to ask follow-up questions, with the system maintaining conversational context to provide relevant responses.
- **Filtered Search Capability:** Users can implicitly or explicitly guide the search by mentioning project numbers, customer names, or other metadata, leading to more targeted and relevant information retrieval.

Document Generation Deliverables:

- **Automated Document Creation:** Generation of fully-populated, professionally formatted .docx documents based on user requests and pre-defined templates.
- **AI-Powered Content:** Document sections designated by <prompt> tags are filled with relevant, context-aware content generated by the advanced RAG engine.
- **Email Delivery:** Finalized documents are automatically emailed to the requesting user, providing a convenient and asynchronous delivery mechanism.
- **Auditable Document Archive:** Generated documents are stored in a structured manner in GCS, providing an audit trail and repository.

Image Digitization Deliverables:

- **Entity Detection:** Pinpointing symbols, vessels, and specialized line types within Piping and Instrumentation Diagrams (P&IDs).
- **Entity Classification:** Categorizing identified symbols, vessels, and specialized line types.
- **Line Detection:** Recognizing continuous, segmented, and specialized line variations.
- **Graph Construction:** Developing a graphical representation to show the relationship between symbols.

5. Ingestion Pipeline

5.1. Introduction

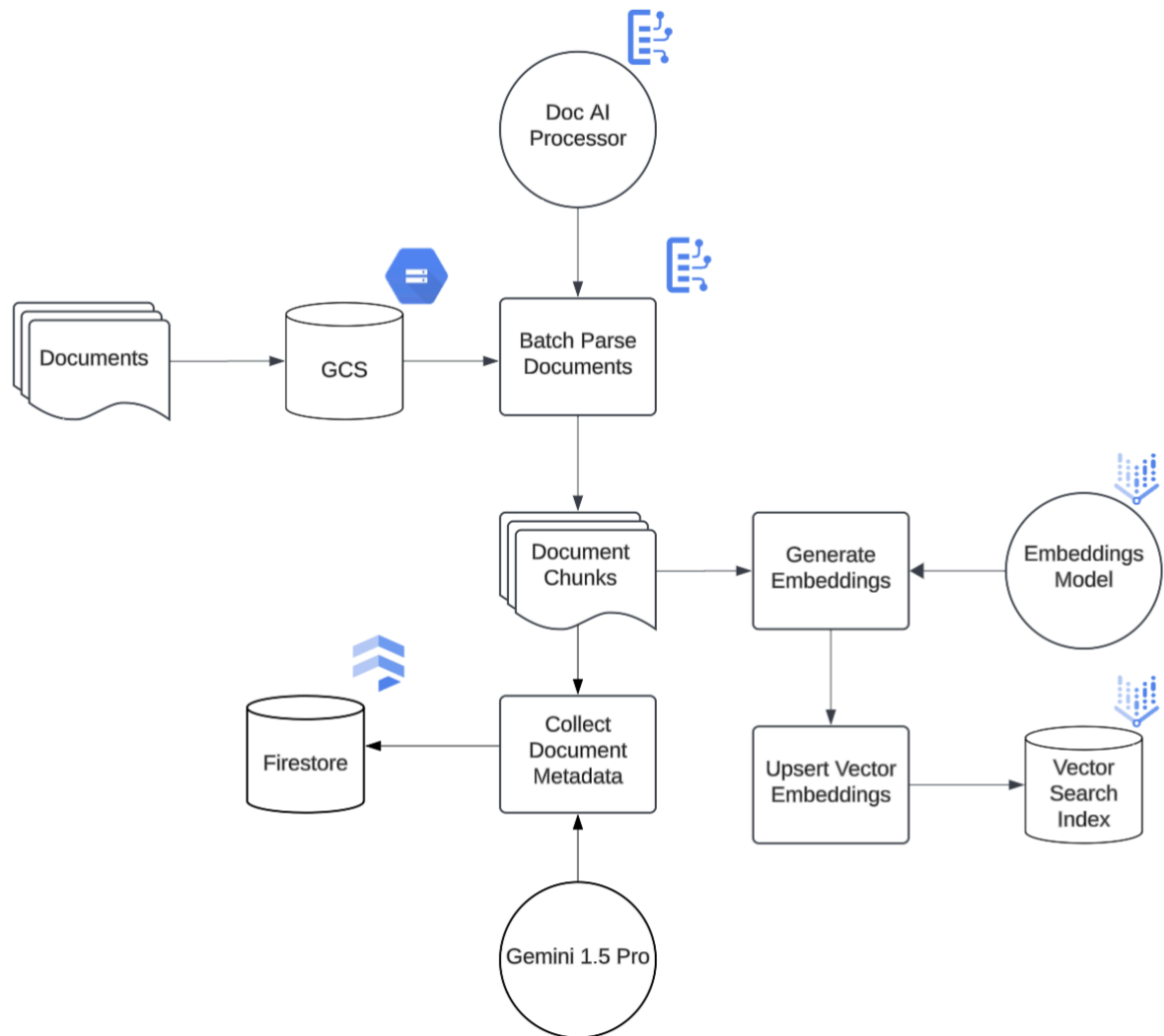
The Ingestion Pipeline is the automated, scalable process responsible for identifying, processing, and indexing documents to make them discoverable by our Retrieval-Augmented Generation (RAG) system. Its fundamental goal is to transform raw, unstructured data from GCS into a structured and

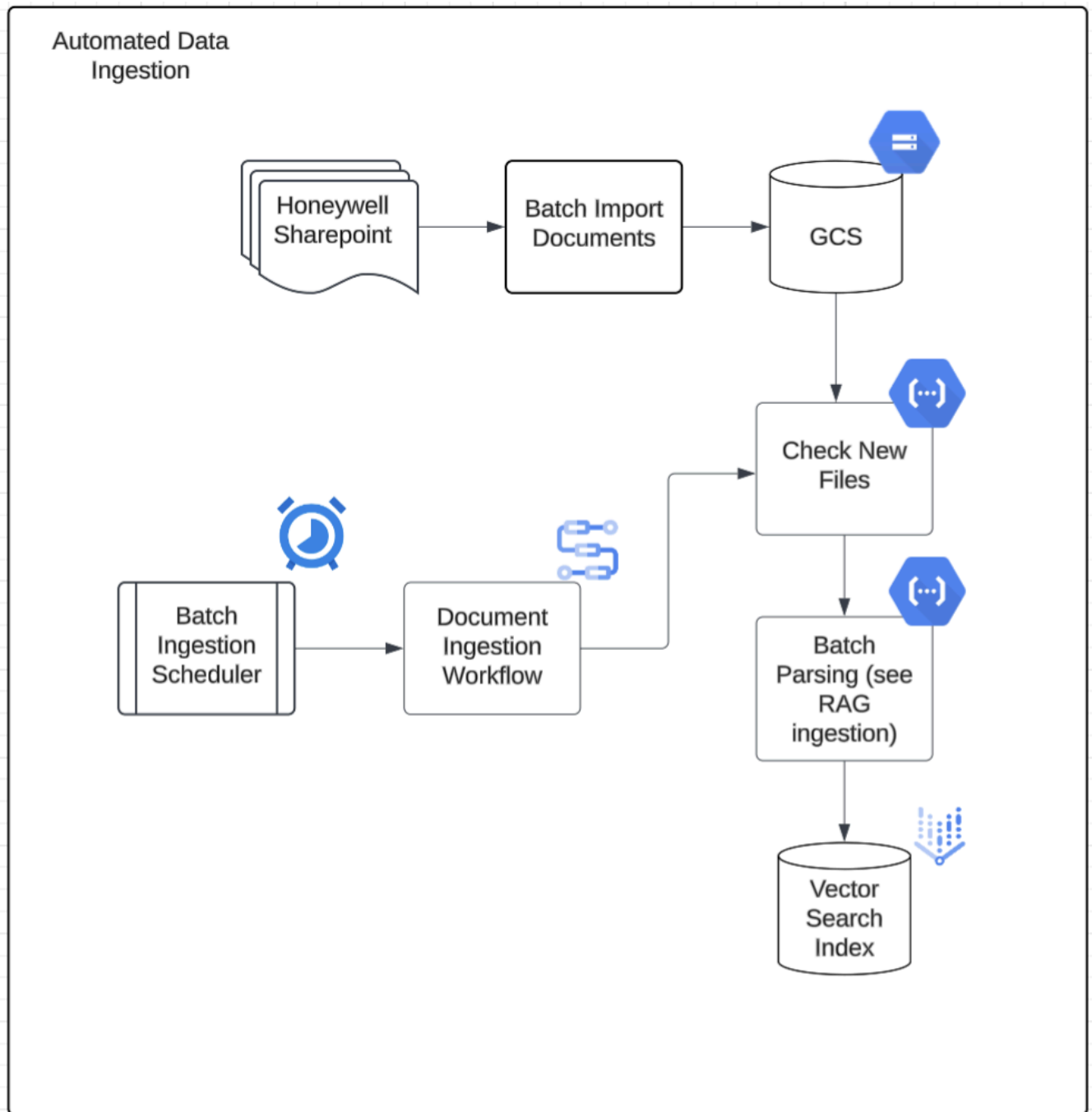
semantically rich format suitable for high-performance vector search.

This is achieved through a multi-stage, orchestrated workflow that handles file validation, conversion of proprietary formats (like DOCX, PPTX) into a standardized PDF format, intelligent parsing using advanced layout models, metadata enrichment from a centralized Firestore database, and the generation and indexing of vector embeddings. The entire pipeline is designed for robustness, featuring comprehensive error handling, status logging, and parallel processing to manage high-volume data streams efficiently. Two parallel entry points exist (`spyder-incoming-doc-checker.yaml` for text extraction and `spyder-incoming-doc-checker-doc-gen.yaml` for document generation sources), both of which converge into a single, unified processing workflow, ensuring consistency while accommodating different data sources.

5.2. Architecture

RAG Data Ingestion Process





The pipeline's architecture is built on a serverless, event-driven model using Google Cloud Platform, designed for high scalability and low operational overhead. It decouples specific tasks into independent microservices, orchestrated by a central workflow engine.

The core components and their specific roles are:

- **Cloud Schedulers:** Trigger the initial `spyder-incoming-doc-checker` and `spyder-incoming-doc-checker-doc-gen` workflows on a recurring schedule, initiating the entire

ingestion process.

- **Google Cloud Storage (GCS):** The central data repository. It hosts:
 - Source documents in their native formats.
 - Batch files (`.txt` chunks) containing URLs of new documents to be processed.
 - Intermediate artifacts, such as PDFs converted from Office formats.
 - Final output logs detailing successful and failed ingestion operations.
- **Cloud Workflows** (`spyder-doc-ingestion-wf-v1.yaml`): The heart of the pipeline, acting as the primary orchestrator. It defines the entire business logic, including the sequence of steps, parallel execution blocks with concurrency limits, error handling using `try/except` blocks, and data flow between services. It passes state and variables (e.g., lists of URIs) between steps.
- **Cloud Functions:**
 - **File Checker** (`spyder-check-new-files-v1`): Scans GCS prefixes for new files based on their creation time (`blob.time_created >= cutoff_time`) and intelligently filters out URIs from document generation output folders to prevent ingestion loops.
 - **Batch Reader** (`spyder-read-files-to-process`): A simple function to retrieve the content of the oldest URI batch file from GCS and then delete it, ensuring that each batch is processed exactly once.
 - **File Splitter** (`spyder-split-large-files`): Checks PDF file sizes and splits documents larger than 15MB into smaller PDFs, prepending the first two pages to subsequent chunks to preserve context.
 - **Document Ingestor** (`spyder-document-ingestion`): The most complex function, responsible for the final stage of processing: retrieving metadata, calling Document AI for parsing, generating embeddings, and upserting the data into Vector Search.
 - **Status Writer** (`spyder-write-processing-status`): Collects results from the workflow execution and writes detailed success and failure logs to GCS, while also triggering an email notification.
- **Cloud Run** (`docx-to-pdf-job-v3`): A containerized service dedicated to file format conversion. It runs a lightweight container with a **LibreOffice** dependency to reliably convert various Office document formats to PDF via a headless command-line interface. This task is isolated to Cloud Run due to its specific external dependency and potentially longer execution time.
- **Document AI:** We leverage the `LAYOUT_PARSER_PROCESSOR`. This processor is crucial as it goes beyond simple OCR, interpreting the document's structure to perform semantic chunking based on layout elements like headings, paragraphs, and tables. This is configured in the `DocAIParser` utility with `include_ancestor_headings=True`.
- **Firestore:** The authoritative source for document metadata. The pipeline queries Firestore using the filename to retrieve essential attributes like `Project_Num`, `GBE`, and `Document Type`, which are then attached to the vectors.
- **Vertex AI Vector Search:** The managed vector index where the final embeddings are stored. The attached metadata enables the RAG system to perform filtered semantic searches, significantly

improving retrieval accuracy.

5.3. End-to-End Flow

The end-to-end process is defined within `spyder-doc-ingestion-wf-v1.yaml` and executes as follows:

5.3.1. File Discovery and Batching:

- The `spyder-incoming-doc-checker.yaml` workflow is triggered by a scheduler. It calls the `check_new_files` Cloud Function, passing bucket and prefix information via environment variables (`INPUT_BUCKET`, `INPUT_PREFIX`).
- The function calculates a `cutoff_time` based on the `windowMinutes` parameter and lists all blobs created after this time.
- A critical filtering step is applied to exclude files within `document_generation` directories that are also in `templates` or `output` sub-folders, preventing the pipeline from re-ingesting its own outputs.
- The resulting list of valid new file URIs is split into smaller chunk files (e.g., 200 URIs per file) and written to the GCS `CHUNK_BUCKET`.

5.3.2. Workflow Orchestration and URI Processing:

- The main `spyder-doc-ingestion-wf-v1.yaml` workflow is triggered. It calls the `readOldestChunkFile` step, which retrieves a batch of URIs.
- The workflow then enters a `parallel` block to process each URI concurrently, with a `concurrency_limit` of 50 to manage resources. Within this block, each file undergoes the following sub-flow, wrapped in a `try/except` block to isolate failures.

Step 1: PDF Conversion (Cloud Run Job)

- A `switch` statement checks the file extension of the `singleUri`.
- **Office Docs:** For formats like `.docx`, `.pptx`, etc., the workflow calls a Cloud Run job (`callConvertJob`). It passes the `INPUT_GCS_URI` and `OUTPUT_GCS_URI` as environment variables. The job downloads the source file, executes `libreoffice --headless --convert-to pdf`, and uploads the resulting PDF. The output path is constructed to mirror the input path (e.g., `gs://in-bucket/folder/file.docx -> gs://out-bucket/folder/file.docx.pdf`).
- **Supported for Conversion:** The following file extensions are supported and will be automatically converted to PDF for ingestion: `.doc`, `.docx`, `.docm`, `.ppt`, `.pptx`,

- `.pptm`, `.xls`, `.xlsx`, `.xism`, `.xlsb` (and their uppercase variants).
- **Supported Directly:** `.pdf` files are supported directly and bypass the conversion step.
- **Unsupported:** Any file extension not on the lists above is considered unsupported by the current pipeline. The workflow will route these files to a failure branch, and they will be logged in the `failedUri` and `failedUriDetailedLogs` reports with the reason "file format not supported".

Step 2: Large File Splitting (Cloud Function)

- The workflow enters a new `parallel` block (`callSplitCF`) to process the list of `pdfUri`s.
- The `spyder_split_large_files` function is called for each PDF. It checks the blob size; if it's $< 15 * 1024 * 1024$, the original URI is returned.
- If the file is larger, the `split_pdf` utility is invoked. It calculates the number of chunks needed and then iterates, creating new PDF files. For every chunk after the first one (if $i > 0$), it prepends the first two pages of the source document (for `p` in `range(min(2, total_pages))`: `writer.add_page(reader.pages[p])`) before adding the pages for the current chunk. This ensures critical context is not lost.
- The URIs of the new chunks are returned and aggregated into the `splitResults` list.

Step 3: Document Ingestion (Cloud Function)

- The workflow enters its final `parallel` processing block (`docIngestion`) with a `concurrency_limit` of 20, iterating over the final list of PDF URIs.
- The `ingest_file` Cloud Function is called for each URI.
 - **Metadata:** It first calls `get_metadata_for_filename` to query Firestore based on the filename, retrieving a dictionary of metadata.
 - **Parsing:** It uses `DocAIParser` to initiate a batch parsing job on Document AI, configured with a `chunk_size` of 1000 characters and `include_ancestor_headings=True`. This is a long-running, asynchronous operation that returns semantically coherent chunks.
 - **Embedding & Indexing:** Once parsing is complete, the function generates embeddings for each text chunk using the `text-embedding-004` model. It then calls `vector_store.add_texts`, which upserts the text, the embedding, and the retrieved Firestore metadata into the Vertex AI Vector Search index.

Step 4: Status Reporting and Completion:

- After the parallel ingestion block completes, the workflow makes a final call to the `callWriteStatus` step.
- The `write_status_files` function receives the `successfulUris`, `failedUris`, and `failedUrisDetailedLogs` lists.
- It creates several `.txt` files in the output GCS bucket:
 - `all_failed_files/`: Contains all URIs that failed at any stage.
 - `supported_failed_files/`: Contains URIs of supported file types that failed during processing.
 - `failed_files_detailed_logs/`: Contains the URI, the stage of failure, and the specific error message for each failure.
 - `success_files/`: Contains all URIs that were successfully indexed.
- Finally, it calls the `send_uri_status_email` utility to dispatch an email notification summarizing the run.

5.4. Key Design Decisions

The design of the ingestion pipeline was guided by the principles of automation, scalability, robustness, and retrieval quality.

- **Orchestration with Cloud Workflows:** Chosen for its native ability to orchestrate complex, multi-service processes with declarative `try/except` error handling, built-in retries for HTTP calls, and robust state management. This is superior to chaining functions via events, which can be difficult to debug and manage.
- **Decoupled Services for Specific Tasks:** We consciously decided to use the best tool for each job. Lightweight, stateless tasks are handled by Cloud Functions. The dependency-heavy file conversion, requiring LibreOffice, is containerized in a Cloud Run job, keeping the Cloud Functions lean and fast to deploy. This separation of concerns improves maintainability and scalability.
- **Semantic Chunking with Document AI Layout Parser:** We chose Document AI's Layout Parser over simple fixed-size or recursive character splitting. This allows us to create chunks that respect the document's semantic structure (paragraphs, headings, tables). This is a critical decision that directly enhances the quality of context provided to the RAG model, leading to more accurate and relevant answers.
- **Context Preservation in Large Files:** The decision to prepend the first two pages of a large document to each of its subsequent chunks is a deliberate strategy to combat context loss. Since initial pages often contain titles, authors, summaries, and other key identifiers, this ensures that even chunks from the end of a document have access to this vital context.
- **Centralized and Extensible Metadata in Firestore:** Using Firestore as a central metadata store, queried at ingestion time, decouples the document's content from its attributes. This allows us to easily enrich vectors with filterable data. This design is highly extensible, as new metadata fields can be added to Firestore and used for filtering without changing the core ingestion pipeline code.

- **Robust and Granular Error Logging:** The pipeline doesn't just track success or failure; it tracks *where* and *why* a file failed. By creating a `failedUriDetailedLogs` list containing the URI, the failed step (`callConvertJob`, `docIngestion`, etc.), and the specific error message, we provide detailed diagnostics that are essential for observability and troubleshooting in a production environment.

6. Text Extraction

6.1 Introduction

Text Extraction, implemented as the `doc_search` tool within the Spyder agent, is the foundational retrieval capability of the entire Spyder (Knowledge Assist) system. Its primary purpose is to perform highly accurate, context-aware Retrieval-Augmented Generation (RAG) across a corpus of over 200,000+ documents. It answers user queries directly by identifying and synthesizing the most relevant text snippets from this vast knowledge base.

The system is engineered to combine the power of semantic understanding with the precision of structured data. It leverages Vertex AI Vector Search for fast nearest neighbor search while applying a complex, multi-faceted filtering system powered by metadata stored in Firestore. These filters, which can be explicitly selected by the user in the UI (e.g., GBE) or implicitly derived from the natural language query by a Gemini model (e.g., Project Name, Opportunity Number), drastically prune the search space, ensuring results are not only semantically similar but also contextually appropriate. The final output is a synthesized answer, grounded in verified source documents provided as citations, which serves as both a direct response to the user and a critical input for other agent tools, such as Document Generation.

6.2 Background

The Spyder agent interacts with a frontend UI through a series of different systems. First, there is the frontend UI which is a React JS application deployed on Azure. This application connects to an Azure deployed agent called "Maintenance Assist", which for the specific deployed tenants associated with Spyder has been configured to have 1 tool that it exclusively calls, effectively forcing it to forward all requests along to the GCP-deployed Spyder agent system. This GCP-deployed Spyder agent system has 2 services: a Token Exchange Service and an Agent Runtime, both deployed to Cloud Run. The Token Exchange Service works by

taking an session ID and verifying the session ID is valid by querying the Maintenance Assist auth API. If the session ID is valid, a GCP JWT is valid and used to perform authentication. Additionally, this token exchange service provides a /docs endpoint which allows the Maintenance Assist agent to understand how to query the Token Exchange API (e.g. what the structure of the payload should look like). The OpenAPI spec is shown below:

```
Python
OPENAPI_SPEC = {
    "openapi": "3.0.0",
    "info": {
        "title": "Spyder Chat Orchestration API",
        "version": "v1",
        "description": "Agent that knows how to handle any user query for Spyder Project Assist."
    },
    "servers": [
        {"url": "https://projectassist-beta.honeywell.com"}
    ],
    "paths": {
        "/chat_v2": {
            "post": {
                "summary": "Agent that knows how to handle any user query for Spyder Project Assist.",
                "description": "Agent that knows how to handle any user query for Spyder Project Assist.",
                "operationId": "chat_v2",
                "requestBody": {
                    "required": True,
                    "content": {
                        "application/json": {
                            "schema": {
                                "type": "object",
                                "properties": {
                                    "query": {
                                        "type": "string",
                                        "description": "The users chat query."
                                    }
                                }
                            },
                            "required": [
                                "query"
                            ]
                        }
                    }
                }
            }
        }
    }
}
```

```

    },
    "security": [
      {"bearerAuth": []}
    ],
    "responses": {
      "200": {
        "description": "Successful response from the Agent Runtime.",
        "content": {
          "application/json": {
            "schema": {
              "type": "object",
              "description": "The JSON response from the Spyder agent.",
              "properties": {
                "output": {
                  "type": "object",
                  "description": "Contains the input query and the agent's
output.",
                  "properties": {
                    "input": {
                      "type": "string",
                      "description": "The original input query."
                    },
                    "output": {
                      "type": "string",
                      "description": "The Agent Runtime's response."
                    }
                  }
                },
                "required": [
                  "input",
                  "output"
                ]
              }
            },
            "required": [
              "output"
            ]
          },
          "example": {
            "output": {
              "input": "What is the capital of France?",
              "output": "The capital of France is Paris."
            }
          }
        }
      }
    }
  }
}

```

```

    }
  },
  "400": {
    "description": "Malformed request or invalid content type.",
    "content": {
      "text/plain": {
        "schema": {
          "type": "string",
          "example": "Malformed request, received incorrect runtime
parameters"
        }
      }
    }
  },
  "401": {
    "description": "Unauthorized user or missing Authorization header.",
    "content": {
      "text/plain": {
        "schema": {
          "type": "string",
          "example": "Missing Authorization header"
        }
      }
    }
  },
  "500": {
    "description": "Internal server error or error querying the Agent
Runtime.",
    "content": {
      "text/plain": {
        "schema": {
          "type": "string",
          "example": "Error querying Agent Runtime: ..."
        }
      }
    }
  }
},
"components": {
  "securitySchemes": {
    "bearerAuth": {

```

```

        "type": "http",
        "scheme": "bearer",
        "bearerFormat": "base64"
    }
}
}
}

```

It also will query Maintenance Assist and grab chat_history that is available for a given conversation using a conversation ID that is provided as part of the auth header. The full list of fields available as part of the auth header are listed below:

Python

```

session_id = auth_dict["session_id"]
tenant_id = auth_dict["tenant_id"]
agent_id = auth_dict["agent_id"]
chat_id = auth_dict["conv_id"]
xsrf_token = auth_dict["xsrf_token"]

```

Lastly, the token exchange service will then restructure the request payload in a specific structure that is required for the Agent Runtime, and then forward the request over to the Agent Runtime for processing, using the GCP JWT as the auth header. The specific structure of the payload is shown below:

Python

```

headers = {
    "Authorization": f"Bearer {access_token}",
    "Content-Type": "application/json",
}

structured_input = {
    "type": "human",
    "content": input_text
}

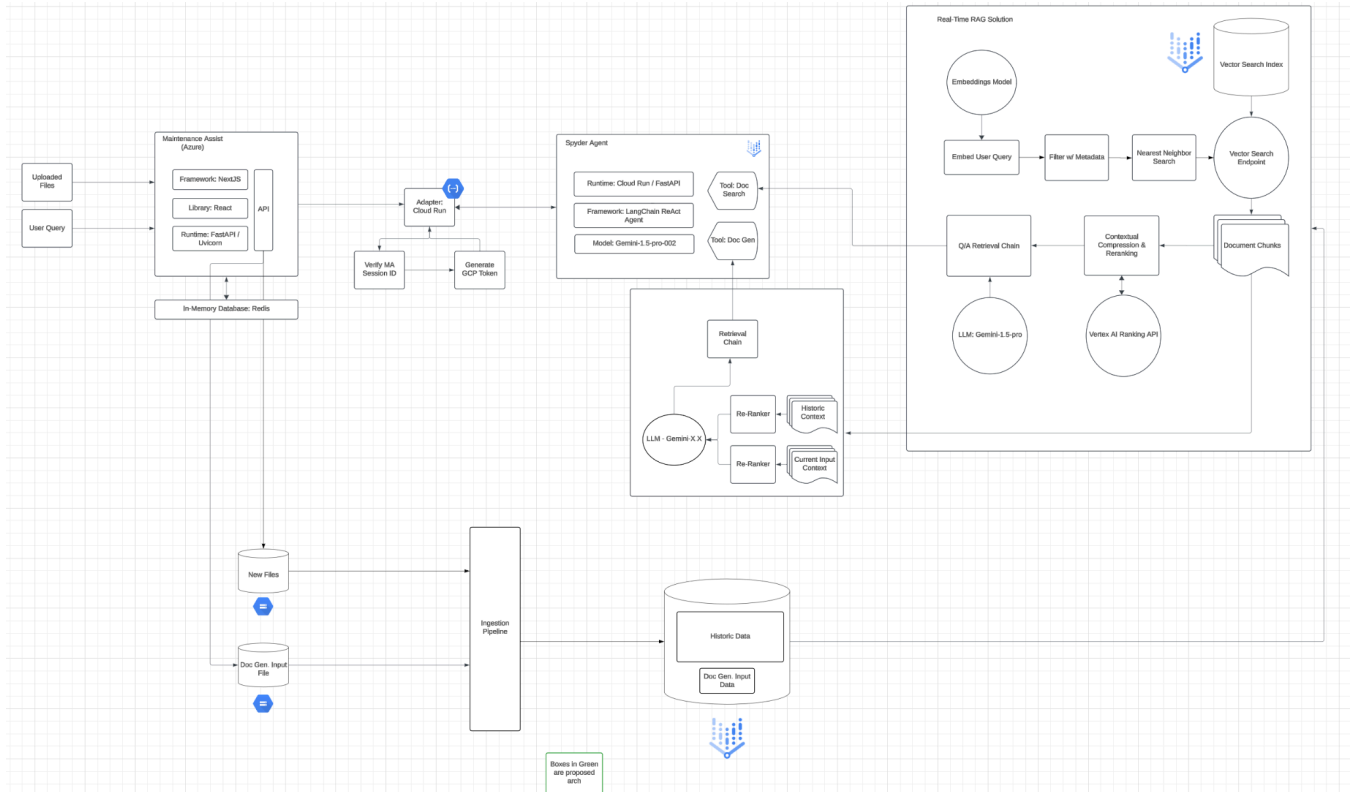
data = {

```

```
"input": {  
  "input": structured_input,  
  "chat_history": chat_history,  
  "config": {"configurable": {"session_id": session_id, "tenant_id": tenant_id}},  
}  
}
```

The Agent Runtime itself is a ReAct LangChain Agent, which has 2 available tools at its disposal: Document Search (text extraction) and Document Generation. Document Generation is covered in a separate section. Document Search will perform RAG across a large number of documents, and is mentioned in detail in the succeeding sections.

6.3 Architecture



The Text Extraction architecture is a real-time, multi-stage pipeline designed for speed, relevance, and accuracy. As illustrated in the "Real-time RAG Solution" portion of the overall system diagram above, it seamlessly integrates language models with specialized Google Cloud services.

The primary components and their roles are:

- **Spyder Agent (Agent Runtime):** The central orchestrator. It receives the structured request from the Token Exchange Service, which includes the user's query, chat history, and session context. Its primary responsibilities are:
 - Hosting the LangChain ReAct Agent, which interprets user intent.
 - Invoking a preliminary LLM call to extract potential filter entities from the user's query.
 - Selecting the doc_search tool when the user's intent is to find information.
- **Filter & Metadata Subsystem:** This logical component is responsible for constructing the precise search constraints. It combines filters from multiple sources:
 - **Explicit UI Filters:** Values passed from the frontend, such as the GBE dropdown, which are passed through the entire request chain.

- **Implicit Query Filters:** Entities like Project Name or Opportunity_Num extracted from the query text using Gemini.
- **Firestore:** The backend for this subsystem, providing fast lookups to validate entities and retrieve associated metadata.
- Specific fields with examples are listed below:

None

```
Customer name: ""(string)
Document Type: "09_ProductManual"(string)
Function: "PM"(string)
GBE: "7fa4b9a2-ca79-4d3f-bc27-fd81de479828"(string)
Location: ""(string)
Name: "ba-fire-D0C-02-130-Digital-Voice-Command-Rev-B.pdf"(string)
Name_without_Ext: "ba-fire-D0C-02-130-Digital-Voice-Command-Rev-B"(string)
Opportunity_Num: ""(string)
Project Name: ""(string)
Project_Num: ""(string)
URL:
"https://prod-edam.honeywell.com/content/dam/honeywell-edam/hbt/en-us/documents/litera
ture-and-specs/datasheets/notifier-aus/ba-fire-D0C-02-130-Digital-Voice-Command-Rev-B.
pdf"(string)
Vertical: ""(string)
gcs_target_url:
"https://storage.cloud.google.com/projectassist_prod/gbe/hbs/product_manuals/ba-fire-D
0C-02-130-Digital-Voice-Command-Rev-B.pdf"
```

- **Real-time RAG Pipeline (The doc_search tool):** A chain of services that executes the core retrieval logic:
 - **Embeddings Model:** A Vertex AI model that converts the user's query into a high-dimensional vector for semantic comparison.
 - **Vertex AI Vector Search:** The core search engine. It performs an approximate nearest neighbor (ANN) search on millions of data points, constrained by the filters provided, to retrieve an initial set of relevant document chunks.
 - **Vertex AI Reranking API:** A crucial refinement step. It re-evaluates the top results from the vector search using a more sophisticated model, re-ordering them based on their contextual relevance to the full query, significantly improving precision.
 - **QA Retrieval Chain (LLM: Gemini 1.5 Pro):** The final synthesis stage. It takes the top-ranked, highly relevant document chunks as context and uses a

powerful LLM to generate a coherent, human-readable answer to the original user query.

6.4 End-to-End Flow

The Text Extraction process is a synchronous flow that begins with the user's query and concludes with a generated answer.

Stage 1: Agent Runtime - Query Parsing and Tool Selection

- 1. Request Ingestion:** The process starts in the Agent Runtime's main service, which receives a POST request at the /chat_v2 endpoint. The request payload, structured by the Token Exchange service, contains the user's input text, chat_history, and configuration including session_id and tenant_id.
- 2. Filter Generation:** Before invoking the agent, the runtime prepares the search filters. It combines:
 - **Explicit Filters:** The GBE ID is retrieved from the session context, which was originally selected by the user in the Azure-hosted React UI.
 - **Implicit Filters:** A dedicated "pre-agent" LLM call (using Gemini) is made to parse the input_text. The model is prompted to extract key entities corresponding to the available metadata fields (e.g., Project Name, Opportunity_Num, Document Type). Any extracted entities are added to the filter set. Below is the current format and the list of columns of the metadata file (As of July 2025)

function	document_type	gbe	project_number	opportunity_id	project_opp_name	file_name	gcs_target_uri	actual_source_url	vertical	location	customer_name
----------	---------------	-----	----------------	----------------	------------------	-----------	----------------	-------------------	----------	----------	---------------

The columns that are being used to filter to identify the right project are opportunity_id, project_number, customer_name and project_opp_name. The latter two leverage fuzzy matching while the first two leverage exact match. In addition, the GBE is used to ensure all search and retrieval only happens within the GBE where the question is being asked. Below are the descriptions and values of the metadata file, as provided by Honeywell. Changes to the existing metadata format will break the existing code and needs to be carefully coordinated with the corresponding code changes.

#	Column/Field Name	Mandatory	Values	Explanation	Comments
1	function	Yes	PM P&E PE PO	Product Manuals Proposal & Estimation Engineering Docs Project Operations	Personas/Functions Classification
2	document_type	No	01_FDS 02_DDS 03_FAT 04_SAT 05_Work Product 06_Customer RFQ_RFB_RFI_Deviation 07_Proposal 08_Estimate 09_Product Manual 10_Other	01_FDS_Functional Design Specification 02_Detail Design Specification 03_Factory Acceptance Test 04_Site Acceptance Test 05_BA_Work Product Manuals_Design Spec 06_Customer RFQ_RFB_RFI_Deviation 07_Honeywell Proposals 08_Honeywell Estimates 09_Product Manual_Offering Doc 10_Other	Document Categories.
3	gbe	Yes	IGS UOP HBS HPS		Respective GBE Values
4	project_number	Depends on functions			Project Number from ERP. Will be mandatory when function=PE.
5	opportunity_id	Depends on functions			Opp ID from SFDC. Will be mandatory when function=P&E and wherever available for function=PE.
6	project_or_opp_name				Project Name or Opp Name based on function = PE or P&E
7	file_name	Yes			Name of the actual file such as .pdf or .docx

8	gcs_target_uri	No			Will be provided once the data is moved to GCP
9	actual_source_url	Yes			Source of the Original File Location. It will be the URL to the file in most cases but not all such as CORA etc.
10	vertical	No			Will be provided if available
11	location	No			Will be provided if available
12	customer_name	No			Will be provided if available

3. Agent Invocation:

- An instance of the SpyderAgentManager is created, which sets up the LangChain ReAct agent with the Gemini-1.5-pro-002 model.
- The user query, the generated filter JSON, and chat history are bundled into a single input string for the agent.
- The agent's LLM analyzes the input (e.g., "In the 'Fire Safety Revamp' project, what are the power requirements for the Digital Voice Command unit?"). Based on its instructions, it recognizes this as a request for information and determines that the doc_search tool is the correct action to take.

Stage 2: Document Search Tool - Retrieval & Filtering

The ReAct agent invokes the doc_search function defined in the agent's tools.

1. **Query Vectorization:** The user's query text is sent to a Google-native embedding model (e.g., text-embedding-004). The service returns a 768-dimension vector embedding representing the semantic meaning of the query.
2. **Filter Construction for Vector Search:** The filter JSON string passed to the tool is parsed. It's transformed into a list of restricted objects compatible with the Vertex AI Vector Search API. Each object is a Namespace key-value pair.
3. **Vector Search Execution:** The tool invokes the vector_search_client.find_neighbors method. It passes the query vector, the restricted list, and a parameter for the number of results to fetch (e.g., num_neighbors=40). The Vector Search endpoint rapidly scans the index, returning the IDs of the document chunks that are both within the filtered metadata scope and semantically closest to the query vector.

Stage 3: Reranking and Synthesis

1. **Contextual Reranking:** The initial 40 document chunks retrieved from Vector Search are not yet in their optimal order of relevance. Their full text, along with the original user query, is passed to the Vertex AI Reranking API. This service re-evaluates the results and returns a new, more accurately ranked list. This step is critical for pushing the most salient information to the top.
2. **QA Context Formulation:** The top K (e.g., K=5) reranked document chunks are selected. Their text content and associated metadata (like Name and URL) are concatenated into a single block of context.
3. **Synthesized Answer Generation:** This context block is passed to a final LLM call, typically within a LangChain QA chain. The prompt is structured as follows:
"Based on the following CONTEXT from company documents, provide a comprehensive answer to the USER'S QUESTION. Cite your sources using the document names provided in the context. An example of this is shown below for illustration purposes:

```
None
CONTEXT:
[Text of chunk 1]
Source: ba-fire-DOC-02-130-Digital-Voice-Command-Rev-B.pdf

[Text of chunk 2]
Source: fire-safety-revamp-project-charter.docx
...

USER'S QUESTION:
[Original user query]"
```

4. **Response Delivery:** The LLM generates a final, synthesized answer. This text is returned by the doc_search tool to the agent, which formats it as the final output and sends it back through the stack to the user's UI.

6.5 Key Design Decisions

The design of the Text Extraction pipeline is guided by a focus on relevance, speed, and intelligence.

- **Improved Search Strategy (Semantic + Metadata Filtering):** Relying solely on semantic search is insufficient with such a large corpus of documents. Metadata filtering is a non-negotiable requirement. This improved search strategy provides the "best of both worlds": the broad, nuanced understanding of semantic search, constrained by the hard, factual boundaries defined by metadata. This dramatically reduces hallucinations and irrelevant results by ensuring the search is performed only on the correct subset of documents from the start.
- **Decoupled Multi-Stage Retrieval Pipeline:** The retrieval process is deliberately broken into stages: fast ANN search, followed by more computationally intensive reranking, and finally LLM synthesis. This layered funnel approach is a key optimization. It allows us to use the fastest-but-less-precise tool (Vector Search) on the entire corpus and reserve the "smarter," slower, and more expensive components (Reranker, Gemini 1.5 Pro) for a small, pre-filtered set of high-potential candidates, balancing cost, latency, and quality.
- **LLM-Powered Filter Extraction:** Using a Gemini model to extract filters like Project Name from natural language makes the system significantly more user-friendly. Users do not need to interact with complex filter menus or remember exact project numbers. They can ask questions conversationally, and the system intelligently translates their intent into the structured queries required by the backend.
- **Tool-Based Architecture in a ReAct Agent:** Encapsulating the entire RAG pipeline as a `doc_search` tool provides critical modularity. The agent is not hardcoded to always search; it reasons and chooses to search. This architecture allows for more complex workflows, such as the agent deciding to first call `doc_search` to gather facts and then passing those facts to the `doc_gen` tool to write a summary.
- **Centralized Metadata in Firestore:** Using Firestore as the metadata store decouples the document metadata from the vector index itself. This is a crucial architectural decision, as it allows for the rapid update of metadata (e.g., changing a project's status) without the need for a costly and time-consuming re-indexing of the document content in Vertex AI Vector Search. Firestore's low-latency key-value lookups are ideal for the fast filter-building needed for each query.

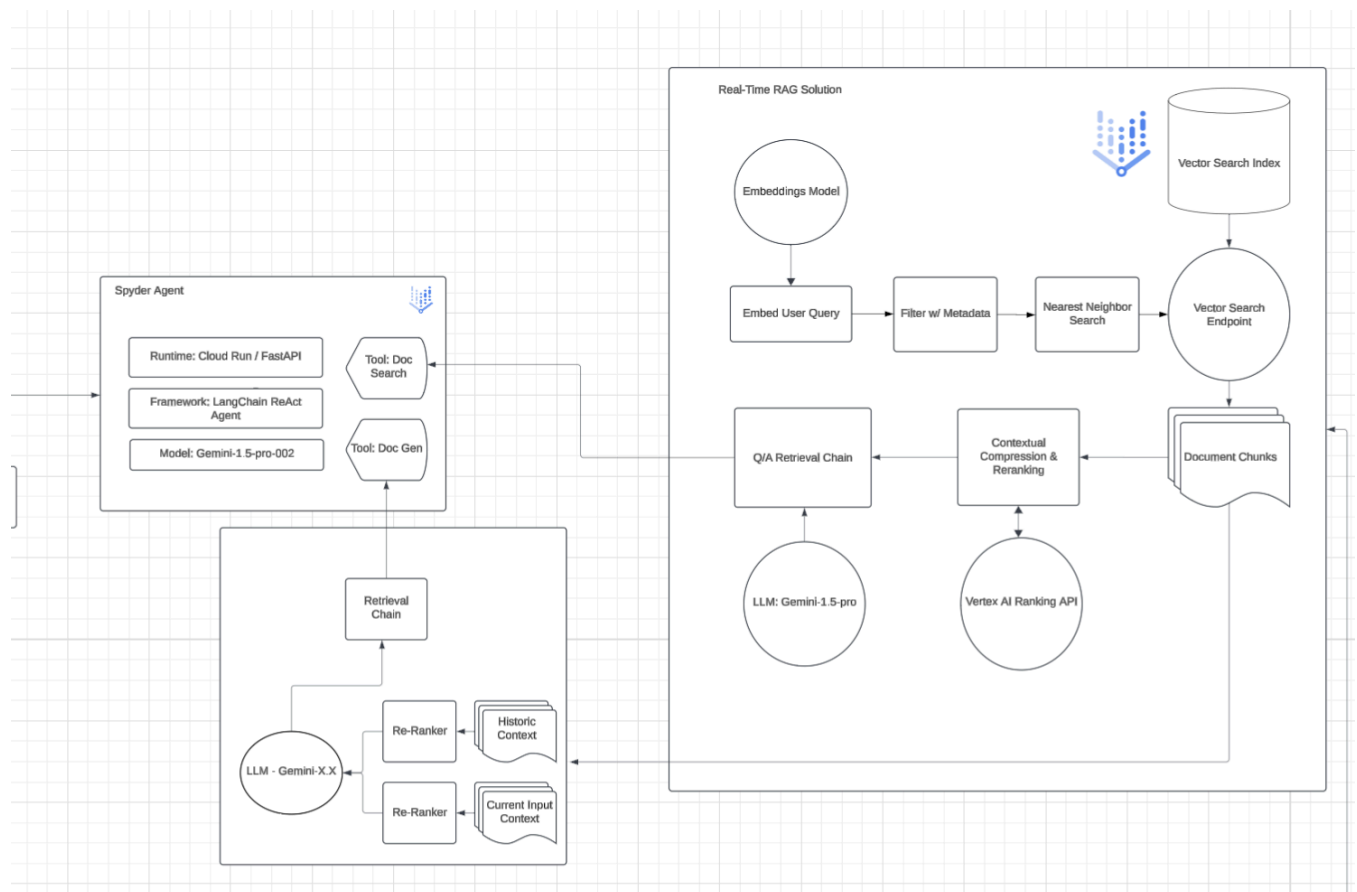
7. Document Generation

7.1. Introduction

Document Generation is a sophisticated, tool-driven capability of the Spyder agent that automates the creation of complex, data-rich documents. It is designed to transform user prompts into fully-realized `.docx` files by populating pre-defined templates with contextually-aware, AI-generated content. This allows users to generate artifacts like functional design specifications, project proposals, or technical summaries with a single command.

The process is architected as an asynchronous workflow, ensuring a responsive user experience. When a user requests a document, the agent immediately acknowledges the task and queues it for backend processing. The system then leverages a powerful, multi-stage pipeline that involves template selection, parallelized content generation via Retrieval-Augmented Generation (RAG), and rich-text formatting, with the final document delivered directly to the user via email.

7.2. Architecture



The Document Generation architecture is designed as a distributed, multi-component system that separates high-level reasoning from the intensive mechanics of document assembly. This decoupled model, illustrated in the provided diagram, ensures resilience, scalability, and maintainability.

The primary components and their roles in this flow are:

- **Spyder Agent (Agent Runtime):** The entry point for the user's request, hosted as a Flask application on **Cloud Run**. Its responsibilities include:
 - Receiving the user's query and session information.
 - Performing initial analysis to extract key entities and create metadata filters.
 - Hosting the **LangChain ReAct Agent**, which determines the user's intent and selects the appropriate tool for the job.
- **Agent Tools** (`tools.py`): A collection of functions the agent can invoke. For this workflow, the `document_generation` tool is key. It acts as a "smart" initiator, preparing the precise arguments required by the backend and triggering the asynchronous process.
- **Cloud Workflow** (`spyder-doc-gen-workflow.yaml`): A lightweight, durable workflow that acts as a message bus. Its sole purpose is to receive a request from the Agent Tool and make an authenticated HTTP POST call to the main generation handler. This decouples the agent from the handler, allowing them to evolve independently and ensuring the request is not lost if the handler is temporarily unavailable.
- **Document Generation Handler** (`spyder-document-generation-handler`): A powerful Cloud Function that performs the core document creation logic. As shown in the "Rational Chain" part of the diagram, its process involves:
 - Loading the correct `.docx` template from GCS.
 - Parsing the template to find content placeholders.
 - Executing the **Prioritized RAG Engine** (`document_search_prioritized`) to generate content for each placeholder. This engine leverages both "Current" and "Historic" context, re-ranks results, and uses an LLM to synthesize information.
 - Injecting the generated, formatted content back into the document.
 - Uploading the final artifact to GCS and initiating email delivery.
- **Error Handling Architecture:** This is a multi-layered process with distinct error handling at both the synchronous, user-facing stage (within the Agent Tool) and the asynchronous, backend stage (the handler):

A) Synchronous Pre-flight Checks (in `agent_runtime/tools.py`): Before the asynchronous workflow is even triggered, the `document_generation` tool performs several critical, synchronous checks. If any of these fail, the process stops immediately,

and a specific error message is returned to the user, preventing a long wait for a task that is destined to fail.

1. **Missing Project Number/Opportunity Number:** This is the most critical validation. The tool explicitly checks if the `search_filter_dict` contains a `project_num` or `opportunity_num`. If it does not, the function returns the user-facing error: "**[Error: Project number or Opportunity number is required to generate a document.]**" and the workflow is never initiated.
2. **Template Not Found:** The `select_template_controlled` utility is called to match the user's request to a template file. If it cannot find a confident match (e.g., the fuzzy search score is too low and LLM fallback fails), it returns `None`. The tool checks for this and returns the error: "**[Error: Could not identify the correct document template based on your request.]**".
3. **GBE Mismatch:** The tool checks if the `tenant_id` (GBE) from the user's session matches the GBE associated with the project number in Firestore. If `get_gbe_from_firestore` returns a GBE that does not match the user's `tenant_id`, the function returns the error: "**[Error: GBE of project does not match with user's GBE.]**". This prevents users from one business unit from generating documents for projects in another.

B) Asynchronous Handler Failure (Backend): If all synchronous pre-flight checks pass, the asynchronous workflow is triggered. Error handling at this stage is designed to be robust and to provide clear feedback for a long-running task.

1. **Workflow Level:** The `spyder-doc-gen-workflow.yaml` wraps the call to the handler in a `try/except` block. If the handler function returns a server-side error (5xx) or times out (exceeding the 30-minute workflow HTTP timeout), the workflow's `except` block will catch the exception. The workflow can then be configured to retry or to trigger a failure notification.
2. **Application Level:** Inside the `document_generation_task_handler` function itself, the entire logic is wrapped in a `main try/except` block. If any part of the process fails internally (e.g., loading the template from GCS, a RAG query fails), the exception is caught. The handler will log the detailed error to Cloud

Logging for debugging and then execute the `send_failure_email` utility function. This proactively sends an email to the requesting user, informing them that their document generation task has failed and including the specific error details. This ensures the user is not left waiting indefinitely for a document that will never arrive.

7.3. End-to-End Flow (Detailed Analysis)

The generation process is a sequence of precise, automated steps, beginning with the user's query and ending with an email containing the final document.

Stage 1: Agent Runtime - Query Parsing and Tool Selection

The process initiates within the `query()` function in `workflows/agent_runtime/main.py`.

1. **Request Handling:** The Flask application receives an `application/json` POST request. The user's query, email (from `session_id`), and `tenant_id` are extracted from the JSON body.
2. **Filter Generation:** The function `get_search_filter` is invoked to analyze the `input_text`. This utility, found in `workflows/agent_runtime/utils.py`, performs a prioritized entity extraction:
 - It first searches for project/opportunity numbers using `extract_project_numbers_controlled`. If found, it validates them against the `qa-document-metadata` collection in Firestore using `match_project_numbers_with_firestore`.
 - If no numbers are found, it proceeds to look for project names using `extract_entity_name_confidently` and `match_name_with_firestore`, which use a combination of keywords and fuzzy matching (`rapidfuzz`). This priority ensures that specific, numeric identifiers are preferred over potentially ambiguous names.
3. **Agent Invocation:**
 - An instance of `SpyderAgentManager` is created. The `create_spyder_agent` method constructs a `ChatVertexAI` LLM, defines a `PromptTemplate` instructing the agent on how to use tools, and creates a LangChain ReAct agent using `create_react_agent`.
 - A composite string, `agent_input_str`, is created using a specific `|||` delimiter to bundle the user's query, the filter JSON, the user's email, and chat history into a single input for the agent.
 - The agent executor is invoked. The LLM, guided by the ReAct prompt, analyzes the query (e.g., "Create a proposal document...") and determines that the `document_generation` tool is the correct action to take.

Stage 2: Agent Tool - Argument Preparation and Workflow Trigger

The agent invokes the `document_generation` function in `workflows/agent_runtime/tools.py`.

1. **Input Validation:** The function first parses the delimited `input_str` back into its components. It then performs a critical validation: `if not search_filter_dict or ('opportunity_num' not in search_filter_dict and 'project_num' not in search_filter_dict): return "[Error: ...]"`. This makes a project or opportunity number a mandatory requirement for initiating document generation.

2. **Template Selection:** It calls `select_template_controlled(query)` to find the correct `.docx` template. This utility in `utils.py` uses a robust two-stage process: first, an LLM extracts a candidate name from the query (e.g., "proposal hec all template"); second, `rapidfuzz.process.extractOne` matches this candidate against the list of files in the `GCS_TEMPLATE_FOLDER` with a score cutoff of 90 for high confidence.

3. **Workflow Payload Creation:** An `args` dictionary is assembled, containing `projectName`, `recipientEmail`, `templateName`, and the `searchFilter` JSON string.

4. **Workflow Execution:** Using `executions_v1.ExecutionsClient()`, the tool triggers the `qa-spyder-etl-doc-gen-workflow` by calling `client.create_execution`. It returns a confirmation message to the user, completing the synchronous part of the flow.

Stage 3: Document Handler - Generation and Delivery

The `spyder-document-generation-handler` Cloud Function is triggered. Its logic is primarily within `utils/document_handler.py`.

1. **Template Loading:** `document_generation_task_handler` loads the specified `.docx` template from GCS into an in-memory `docx.Document` object.

2. **Prompt Extraction:** The code performs a detailed parse of the document:

- It iterates through all paragraphs (`doc.paragraphs`) and table cells (`table.cells`).
- It uses a case-insensitive regex, `re.search(r"<[Pp]rompt>", text)`, to find prompt tags.
- The text content between the tags is extracted and stored. The original paragraph's content is then cleared (`p_elem.clear_content()`) and replaced with a unique placeholder, `f"<placeholder_{uuid.uuid4()}>"`, to bookmark the location.

3. **Parallel Content Generation (RAG):**

- A `concurrent.futures.ThreadPoolExecutor` is used to process all unique prompts in parallel for efficiency.
- Each thread executes `_thread_search_for_generation`, which in turn calls the `document_search_prioritized` RAG function. This function, as depicted in the "Rational Chain" diagram, retrieves context from "Current" and "Historic" vector indexes, re-ranks the results for relevance, and uses a Gemini 1.5 Pro LLM to generate a comprehensive answer based on the prioritized context.

4. **Content Injection and Formatting:**

- The code iterates through the document again, this time looking for the `placeholder_` IDs.
- When a match is found, the corresponding AI-generated answer is retrieved.
- The function `add_markdown_to_paragraph(paragraph, generated_answer)` is called.

This utility is a mini-parser that first checks for block-level markdown (like `## Heading` or `* List item`) and applies the appropriate `.docx` paragraph style (e.g., `paragraph.style = 'List Bullet'`). It then processes inline markdown (like `**bold**`) by creating new `run` objects with the correct character formatting (`run.bold = True`).

- For traceability, the injected content is wrapped in `<generated>` tags and colored green.

5. Finalization and Delivery:

- The final, populated `.docx` object is uploaded to a structured path in GCS: `{DOCUMENT_GENERATION_FOLDER}/{gbe_folder}/output/{opportunity_num}/{generated_file_name}`.
- The `send_email` utility is called to dispatch an email to the `recipientEmail` with the generated document as an attachment.

7.4. Key Design Decisions

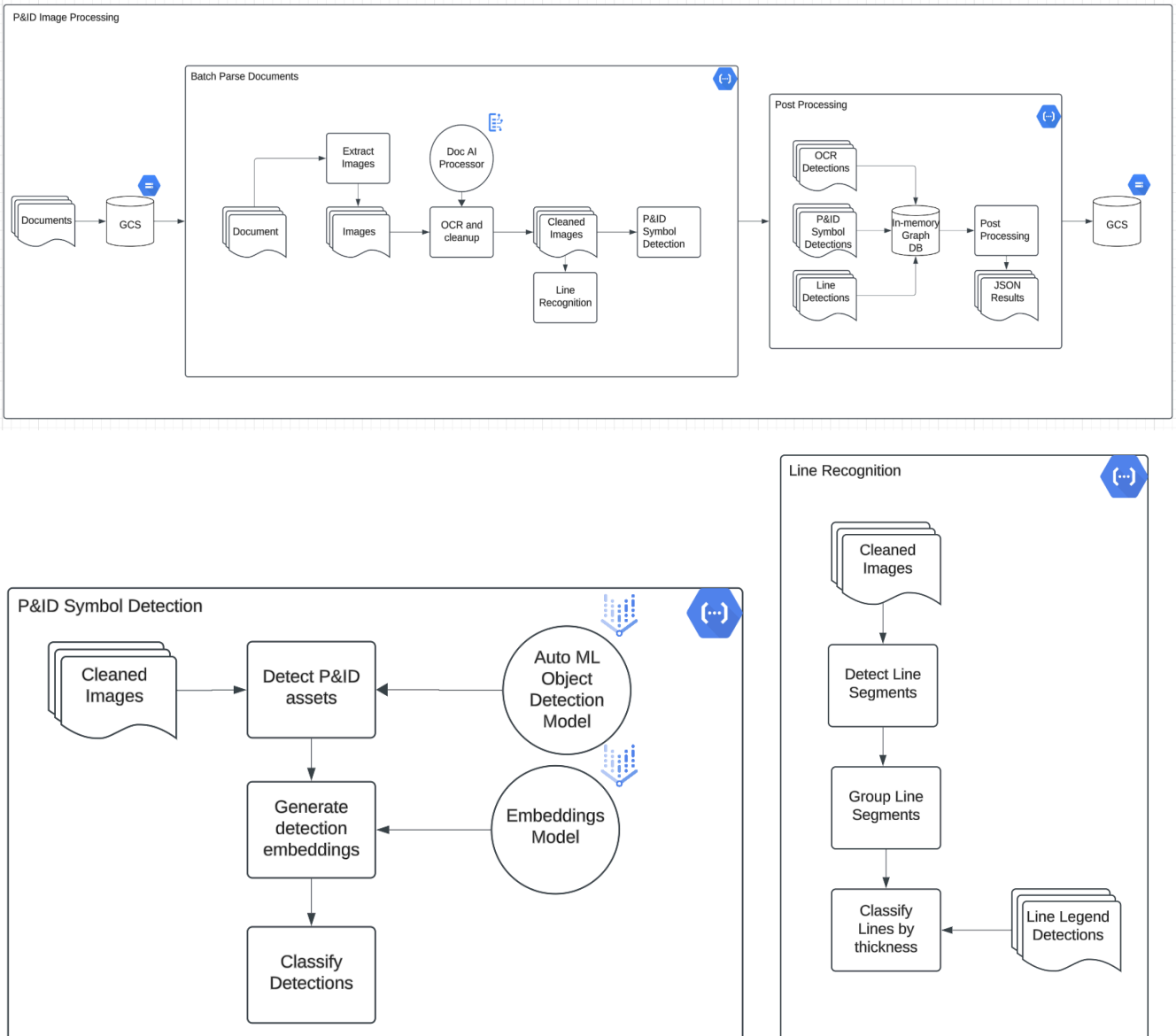
The Document Generation pipeline's design is underpinned by several strategic choices aimed at creating a robust, high-quality, and user-friendly system.

- **Asynchronous-First Architecture:** The immediate hand-off from the synchronous agent to an asynchronous workflow is a critical design choice. It ensures the user receives an immediate response and prevents HTTP timeouts for a process that can take several minutes to complete, providing a superior user experience.
- **Decoupled Agent-Workflow-Handler Pattern:** This three-layer architecture provides strong separation of concerns. The Agent focuses on NLU and tool selection. The Workflow acts as a durable, retry-capable message queue. The Handler encapsulates all the complex document processing logic. This makes the system easier to test, debug, and evolve.
- **Dynamic, Code-Free Templating:** Using simple, human-readable `<prompt>` tags within standard `.docx` files empowers non-technical users to design and modify document templates. They can control the structure, boilerplate text, and locations for AI-generated content without writing a single line of code.
- **Maximized Performance with Parallel RAG:** Generating content for each prompt sequentially would be prohibitively slow. The use of a `ThreadPoolExecutor` to process all RAG queries concurrently is a key performance optimization that dramatically reduces the total document generation time.
- **High-Quality Content through RAG Engine Reuse:** By leveraging the same sophisticated `document_search_prioritized` RAG engine that powers Q&A, we ensure that the content is not just arbitrary text but is grounded in the most relevant, up-to-date, and contextually-aware information available in our vector stores.
- **Rich Formatting via Markdown-to-DOCX Conversion:** The `add_markdown_to_paragraph` utility is a cornerstone feature. It allows the LLM to output content using standard, easy-to-generate markdown. The handler then translates this into native `.docx`

formatting, including styled headings, bulleted/numbered lists, and bold/italic text, resulting in polished, professionally-styled documents that are ready for immediate use.

8. P&ID Image Recognition

8.1 Architecture



Architecture Overview: The PID (Piping and Instrumentation Diagram) digitization process utilizes a sophisticated pipeline, central to which is the processing of PID information from images into a structured digital format. The primary components that facilitate this transformation are detailed below:

1. Entity Detection and Classification:

- Purpose: This initial stage focuses on identifying and categorizing distinct elements within the PID images. These elements include symbols, vessels and special line types indicating different connections and flows.
- Methodology: Object detection models, specifically trained using AutoML, are employed for the initial detection. To ensure precision and efficiency, three separate models are utilized, each tailored to recognize a particular type of entity (symbols, vessels, or special lines).
- Deployment and Scalability: These trained models are deployed on VertexAI endpoints. This deployment allows for dynamic scaling of compute resources, ensuring that each model can be individually scaled based on its specific workload demands. This flexibility is crucial for optimizing performance and managing computational costs.
- Classification Process: Following the detection, the entities are further classified into more granular categories. This classification is achieved through multimodal embedding similarity search. This approach leverages the unique features of each entity, allowing for accurate identification of specific symbol types, vessel specifications, and special line characteristics.
- Modularity and Extensibility: The separation of object detection and classification tasks offers significant advantages. It allows for the easy addition of new entities to the system. By updating the embedding database with new entity representations, the classification process can be enhanced without necessitating retraining of the entire object detection model. This modularity streamlines the process of system updates and improves its adaptability to evolving requirements.

2. Line Detection:

- Purpose: This step is dedicated to identifying all linear elements present in the PID images. These lines typically represent pipes and other flow paths within the system.
- Methodology: To achieve robust and efficient line detection, the system employs an open-source Line Segment Detector (LSD). Specifically, the linear time LSD available at <https://www.ipol.im/pub/art/2012/gjmr-lsd/> is utilized.
- Key Features: This LSD algorithm is characterized by its efficiency and lack of requirement for manual parameter tuning. Its algorithmic design ensures that all straight lines within the image are detected accurately and rapidly, without the need for extensive configuration.

3. Text Extraction:

- Purpose: The text extraction module focuses on identifying and extracting all textual information present in the PID images. This includes labels, annotations, and specifications that are critical to understanding the diagram.
- Methodology: Text extraction, also known as Optical Character Recognition (OCR), is carried out using Google's DocumentAI service. The DocumentOCR parser, a specific processor type of DocumentAI, is employed for its superior accuracy and ability to handle complex document layouts.
- Patching for Large Images: Given the potentially large size of PID images, a patch-based approach is used to optimize processing. The images are divided into smaller, more manageable patches. OCR is then performed on each patch, and the

extracted text segments are subsequently recombined to reconstruct the complete textual information from the original image. This technique improves both performance and accuracy.

4. Graph Construction:

- Purpose: The final stage involves transforming the detected entities and their relationships into a structured graph representation. This graph captures the spatial relationships and connections between different components in the PID.
- Methodology: The module utilizes Python's shapely and networkx libraries. Shapely is used for geometric operations, which help in determining spatial relationships between symbols and lines. Networkx is used for creating and manipulating the graph data structure, enabling the efficient representation and analysis of the connections between symbols.
- Outcome: The resulting graph serves as a digital representation of the PID, capturing the essential connections and relationships between the various components. This representation facilitates further analysis, automation, and integration with other systems.

8.2 End-to-End Flow

The image digitization process consists of 4 steps - entity detection and classification, line detection, text extraction and graph construction. The 1st 3 steps are independent of each other while the last step of graph construction depends on the outputs of the previous 3 steps.

Step 1 - Image OCR using Document AI

Overview

This section details the orchestration process for executing Optical Character Recognition (OCR) on images using Google Cloud's Document AI. The objective is to automate the extraction of textual information from images while ensuring accuracy and efficiency throughout the process.

Process Flow

- 1. Image Acquisition:**
 - The system accepts various image formats as input. Each image is read into the processing pipeline as binary content to facilitate interaction with Document AI.
- 2. Image Binarization:**
 - The images undergo conversion to grayscale and are then binarized. This process enhances the distinction between text and background, improving the accuracy of subsequent OCR operations. This is done in the `binarize_image` function, which applies a threshold to convert pixels based on brightness.
- 3. Splitting Image into Patches:**

- Large images are divided into smaller, more manageable patches using the `split_image_into_patches` function. This is crucial for handling images that exceed the size limits for processing by Document AI:
 - Each patch includes an overlap of 200 pixels to ensure continuity in text recognition across adjacent patches.
- 4. **Document Processing via Document AI API:**
 - Each image patch is processed using the `process_document` function. This function:
 - Creates a `DocumentProcessorServiceClient` with the appropriate API endpoint based on region.
 - Constructs a request to send the image content as a `RawDocument` to Document AI for processing. The API processes the image and returns structured data containing recognized text and accompanying metadata.
- 5. **Text and Token Extraction:**
 - The `extract_text_and_tokens` function retrieves text annotations from the processed document:
 - It captures position data (bounding boxes) for each identified text token.
 - The algorithm also checks for image transformations (e.g., rotations), ensuring accuracy in mapping coordinates back to the original image.
- 6. **Annotation Mapping:**
 - The `remap_annotations_to_base_image` function converts the coordinates of extracted text from the local patch frame to the original image's coordinates. This ensures that annotations reflect their actual positions in the full image.
- 7. **Merging Text Annotations:**
 - The system consolidates overlapping text segments using `merge_and_normalize_annotations`. This process:
 - Merges bounding boxes based on their proximity.
 - Combines text content to form coherent strings, preserving reading order and readability.
 - Ensures that duplicate texts are handled, providing a clean output for further analysis.
- 8. **Result Visualization:**
 - The `draw_white_mask` function is utilized to visually highlight detected text areas on the original image, allowing users to easily see where text was extracted.

Output

The final outputs of this step includes:

- Processed images with highlighted text areas, providing a visual representation of OCR results.
- A data structure containing extracted text with their bounding box coordinates, facilitating downstream processing or analysis.

Step 2 - Entity detection and classification

Overview

This section details the process for detecting and classifying entities within Piping and Instrumentation Diagrams (PIDs).

Process Flow

1. **Image Splitting:**
 - PID images are segmented into smaller patches to enhance model prediction robustness and circumvent file size constraints of the model prediction endpoint (1.5MB limit).
2. **Patch List Creation:**
 - A list of image patches is generated for parallel processing via multithreading. Patches are created with overlaps: 100 pixels for symbols and line types, and 500 pixels for vessels. Overlap ensures entities at patch boundaries are captured.
3. **Image Encoding:**
 - Images are converted to base64 format, as required by the VertexAI endpoint, using the ``encode_image_to_base64`` function.
4. **Entity Detection:**
 - Symbols and special line types are identified using dedicated models on the VertexAI endpoint. Bounding boxes and confidence scores are returned for each prediction. This is facilitated by the ``predict_image_b64_symbol_detection``, ``predict_image_b64_vessel_detection``, and ``predict_image_b64_line_type_detection`` functions.
 - VertexAI's batch prediction service is used for vessel identification to circumvent the 1.5MB file size limit imposed on online prediction endpoints by AutoML models.
5. **Prediction Merging:**
 - Detected entities from patches are consolidated using the ``merge_patch_detections`` function. Overlapping detections are merged if the intersection over union area exceeds a predefined threshold. Merging also occurs if the intersection area with respect to one detection surpasses a certain threshold.
6. **Entity Classification:**
 - Entities are categorized based on the closest matching image embeddings. An embeddings database, stored in separate pickle files for symbols, vessels, and special lines, is utilized. For each detection, an embedding is generated and compared with the database embeddings. Classification occurs if at least two out of the three closest embeddings belong to the same class with a similarity exceeding 70%.
7. **Multithreading Implementation:**
 - Image entity prediction and classification are executed in parallel for all patches/entities using a thread pool, improving processing speed and resource utilization.

Output

The final result of this process is a set of entity predictions, each including:

- Bounding box coordinates
- Predicted class label

- Confidence score

Step 3 - Line detection and segmentation

Overview

This section outlines the procedures for detecting lines, recognizing segmented lines, and applying post-processing.

Process Flow

1. Line Segment Detection:

- The Line Segment Detector (<https://www.ipol.im/pub/art/2012/gjmr-lsd/>) library detects line segments from an image. It utilizes python bindings from the pylsd-nova library. This tool eliminates the need for manual parameter adjustments and provides a list of line segments including start/end coordinates and thickness. Parameters can be adjusted if needed.

2. Dashed Segments Detection and Clustering:

- All line segments are identified by the Line Segment Detector. Shorter, repeating dashes are combined into a single segmented line (chain) using the `'detect_dashed_lines'` function in the `'segmented_line_detection'` module.
- This module consolidates dashes using six configurable parameters:
 1. Angle difference: Maximum angle difference allowed between segments in a chain.
 2. Dash length range: Lengths within this range are considered for chain creation.
 3. Dash size range: Ensures consistent dash length within a chain.
 4. Dash gap range: Ensures consistent gap between dashes in a chain.
 5. Dash offset threshold: Checks if dashes align within a threshold distance.
 6. Candidate search distance: Dashes within this distance are considered for chain growth.

3. Line Segment Filtering:

- Segments that form chains are removed from the original line segment list, leaving continuous and segmented lines. This is performed by the `'update_segments'` function in the `'segmented_line_detection'` module.

4. Merging Close Segments:

- Lines with endpoints within a configurable threshold are merged using the `'merge_close_segments'` function in the `'line_detection'` module.

5. Line Segment Classification:

- Lines are classified as thick or thin based on thickness values from the Line Segment Detector. Lines 4 pixels thick or less are thin; others are thick. This is configured via `'get_class_definitions'` and `'classify_line_segments'` functions in the `'line_detection'` module.

Output

The process yields a filtered list of line segments, including start/end coordinates, line thickness (pixels), and line classification (thick/thin).

Step 4 - Graph Construction Process

Overview

This section outlines the process of building a graph that represents the relationships and connections between symbols in a diagram.

Process Flow

1. **Geometric Conversion:** Converts all symbol bounding boxes into shapely geometries (Polygon, LineString, and Point) for efficient and simplified operations.
2. **Symbol Categorization:** Isolates flow direction symbols, as these are identified by the symbol detection model but excluded from the final graph.
3. **Line Refinement:**
 - Removes lines shorter than 40 pixels to eliminate noise from text elements.
 - Merges collinear lines detected as duplicates (within 10 pixels distance and 5 degrees angle difference). These thresholds are adjustable.
4. **Special Line Type Update:** Detects overlaps between special line type symbols and line segments, and updates the line type accordingly.
5. **Graph Assembly:**
 - Populates a networkx graph with all symbols and vessels as nodes.
 - Creates edges between nodes that are connected by line segments.
 - Handles line intersections by introducing temporary intersection nodes and edges.
 - Removes the temporary intersection nodes, then establishes direct edges between previously connected neighbor nodes, preserving connectivity through intersection points.

Output

The result is a networkx graph, stored as a JSON file, where nodes represent symbols and edges represent their connections. This JSON output is compatible with networkx graph formats for reloading.

8.3 Key Design Decisions

- **Model Differentiation for Enhanced Accuracy:** Employing distinct models for symbols, vessels, and specialized lines improves prediction robustness and precision. This strategy also enables independent model retraining as needed.
- **Object Classification via Detection and Embedding:** Utilizing object detection to establish boundaries, followed by embedding-based similarity search for classification, allows for rapid classification updates. This eliminates the need for full model retraining with each new symbol class.
- **Model-Free Line Detection for Deterministic Results:** Implementing a model-free method for

line detection ensures deterministic line segment identification, reducing the need for extensive model management.

- **Image Patching for Scalability:** Dividing images into patches for object detection and text extraction facilitates scalability for large images without compromising output quality or accuracy.
- **Parallel Processing for Efficiency:** Parallelizing object detection, classification, and text extraction optimizes processing time and resource utilization, especially given the repetitive nature of these tasks on each patch.
- **Event-Triggered Pipeline for Cost-Effectiveness:** An event-based trigger initiates the pipeline upon image upload to the source Google Cloud Storage (GCS) bucket, with output saved to the target GCS bucket. This asynchronous architecture is suitable for non-real-time PID JSON outputs, reducing costs and enabling parallel processing of multiple images via Cloud Run jobs.

9. Limitations

9.1 Token limit

This solution leverages Gemini 1.5, an advanced AI model with a maximum context window of 1 million tokens. A 'token' roughly corresponds to 4 characters of English text. This limitation restricts the model's ability to process extremely long documents or maintain extensive conversation histories. Solution gracefully handles this limit, by keeping an active count of tokens for the retrieved documents and providing a clear error message back to the caller, as listed in Section 9.2 . Also, for follow up questions, since they refer to the same set of documents, the token count remains largely unchanged. Each follow-up question introduces new tokens, but the number is usually much smaller compared to the large number of tokens in the documents. This token limit directly influences system performance. The most significant impact is on **response time**, as a larger context (more retrieved document chunks) increases the latency of the AI's response. While the large window makes it a low risk, the limit can also affect **accuracy** if the context must be truncated, potentially omitting key information. The token limit itself does not impact **repeatability**, as the system's deterministic settings (temperature 0.0) ensure the same input yields the same output.

9.2. End-to-End Processing Time

The processing time for user requests varies significantly between the two main functions:

Text Extraction (Q&A): The end-to-end time for answering a query is typically fast but is dependent on the sequential execution of the RAG chain. This includes the latency of the vector search, the re-ranking step, and finally, the LLM generation time. While generally responsive, complex queries that

require retrieving and processing a large amount of text can lead to noticeable delays.

Document Generation: Generating a document is a much more intensive process. The total time is a function of how many <prompt> tags are in the selected template, as each one triggers a full RAG query. To mitigate this, the spyder-document-generation-handler is designed for parallel processing. It uses a ThreadPoolExecutor to execute all RAG queries for a given template concurrently. This design significantly reduces the overall processing time compared to a sequential approach, but generating documents with dozens of prompts can still take several minutes.

9.3. Quality of Dataset and Model Accuracy

The system's accuracy is fundamentally tied to the quality of the source documents in our corpus. The principle of "garbage in, garbage out" applies directly.

- **Document Quality:** The effectiveness of the Document AI Layout Parser and subsequent chunking is dependent on the quality of the source files. Poorly structured documents, scanned PDFs with low-quality OCR, complex tables, or text embedded within images may lead to suboptimal chunking, affecting the context provided to the RAG model.
- **Model Grounding:** The RAG model is explicitly instructed to answer only based on the retrieved context. If the information does not exist, is outdated, or is incorrect in the source documents, the system will reflect that reality in its answers. It cannot infer knowledge that is not present in its indexed data.
- **Image Quality:** Challenges in accurately extracting information from P&ID diagrams include image quality, resolution, complex layouts, and variations in design standards, which can affect the extraction of symbols, lines, and text. The model may encounter difficulties with proprietary or non-standard symbols and annotations if they are not included in its training data, potentially leading to incomplete or incorrect digitization. The effectiveness of the image recognition model relies significantly on the diversity and quality of its training dataset; a lack of representative data for specific P&ID types or symbols could compromise its performance.

9.4. Quantity and Quality of Metadata

The system's ability to retrieve relevant documents is heavily dependent on the completeness and accuracy of the metadata stored in Firestore.

- **Metadata Dependency:** The `get_search_filter` utility relies on matching entities from a user's query (like a project number or customer name) with entries in the `qa-document-metadata` Firestore collection. If this metadata is missing, incomplete, or inaccurate for a given document, that document will be invisible to filtered searches, even if its content is relevant.
- **"Orphaned" Documents:** Any document uploaded to GCS that does not have a corresponding, correct metadata entry in Firestore is effectively an "orphaned" document. It will not be retrieved by targeted queries that rely on filters, severely limiting its utility within the system.

9.5. User Interface and Discoverability

The system's user interface relies on natural language processing to extract filters from user queries rather than providing structured input methods like dropdown menus. This design choice offers flexibility but introduces limitations.

- **Lack of Explicit Options:** Users have no direct way of knowing which projects, customers, or other filterable entities are indexed in the system <Lack of drop down filters in the UI>. They cannot browse a list of available projects to query against. This can lead to guesswork and potentially "zero-result" queries if the user's terminology doesn't match the metadata.
- **NLP Fragility:** While powerful, the `get_search_filter` NLP model can fail if a query is ambiguous or uses non-standard phrasing for an entity. This makes the system less discoverable and potentially less reliable than a UI with explicit filter options like dropdowns.

9.6. Difficulty in Handling Ambiguous Queries

The system performs best when user queries are specific. Ambiguous or overly broad queries present a significant challenge.

- **Limited Context:** A query like "Tell me about the project" provides very little semantic content for a vector search. If the NLP also fails to extract a specific project number or name from the query or chat history, the search is performed across the entire knowledge base. This often leads to generic or irrelevant results because the system lacks the context to narrow its search.
- **History Dependency:** The `create_history_aware_retriever` is designed to use conversation history to add context, but its effectiveness is limited. The system's performance is significantly better when users provide clear, explicit context in each query rather than relying on the system to infer it from a long conversation.

10. Observability

This section is designed to provide guidance around identifying and measuring the performance and availability of common failure points associated with Honeywell solution. The content in this document will provide approaches and suggestions around common observability patterns and tools which can be used to measure and report on the health of the solution leveraging GCP native resources.

10.1 Cloud Logs

A robust and multi-faceted logging strategy is implemented across the Spyder Project Assist system to ensure comprehensive observability, facilitate real-time debugging, and provide long-term audit trails. The strategy leverages two primary Google Cloud services: **Cloud Logging** for structured, real-time log exploration and alerting, and **Google Cloud Storage (GCS)** for persistent, batch-oriented logging and traceability.

Ingestion Pipeline

The Ingestion Pipeline has the most comprehensive logging due to its multi-step, asynchronous nature. Observability is achieved through a combination of workflow-level logging and detailed application logs. The `spyder-doc-ingestion-wf-v1.yaml` workflow explicitly uses `sys.log` calls at key steps to trace its progress directly within the workflow's execution history. More importantly, the pipeline culminates in the `callWriteStatus` step, which invokes the `spyder-write-processing-status` Cloud Function. This function creates a detailed, persistent record of the entire batch run by writing several files to a GCS bucket. These include `successful_uris`, `failedUris`, and most critically, `failedUrisDetailedLogs`, which contains the URI, the stage of failure (e.g., `callConvertJob`, `docIngestion`), and the specific error message for each failed file. Furthermore, every Cloud Function and Cloud Run job within the pipeline (from `spyder-check-new-files-v1` to `spyder-document-ingestion`) sends all its standard output (`stdout`) and errors (`stderr`) to Google Cloud Logging. This allows engineers to query for real-time logs from any component using the Log Explorer, filtering by function name or request ID to debug specific file-processing issues.

Text Extraction (Q&A)

Observability for the Text Extraction (Q&A) capability is centralized within the Agent Runtime. The `agent_runtime/main.py` script explicitly initializes the Google Cloud Logging client using `client.setup_logging()`. This enhances standard Python logging by routing logs to Cloud Logging as structured JSON payloads. Throughout the request lifecycle, the application logs critical information, including the received JSON request, the constructed filter payload, and the final generated response. The utility functions in `agent_runtime/utils.py` also contain detailed logging. For instance, `get_search_filter` logs which entities were extracted from the user's query and whether they were successfully matched in Firestore. This provides a clear, queryable trail within Cloud Logging to trace how a user's query was interpreted and what filters were applied, which is invaluable for debugging why a particular answer was or was not generated.

Document Generation

The Document Generation flow has a two-tiered logging strategy. The initial request is handled by the **Agent Runtime**, which, as noted above, logs the query analysis and tool selection process to Cloud Logging. Once the `document_generation` tool triggers the asynchronous workflow, logging shifts to the backend components. The `spyder-doc-gen-workflow.yaml` provides high-level execution tracing. The core of the observability comes from the `spyder-document-generation-handler`. Within `utils/document_handler.py`, a comprehensive trace dictionary is created for every generation request (`trace_data`). This dictionary captures the `trace_id`, query, applied filters, counts of retrieved and re-ranked documents, top citations, and the final generated answer. This entire payload is then written as a JSON file to a dedicated GCS bucket (`GCS_TRACE_BUCKET`), providing a complete, auditable record of each document generation task for later analysis and quality evaluation.

Image Digitization

PID image digitization utilizes Cloud Run for scalable image processing and VertexAI for object detection. Cloud Run jobs efficiently handle compute-intensive image processing, while VertexAI endpoints host machine learning models for object identification. Both services offer critical monitoring: Cloud Run logs detail digitization steps and errors, while VertexAI tracks model availability, latency, and throughput. This comprehensive logging and monitoring ensures an efficient, reliable, and traceable digitization pipeline.

10.2 Cloud Run Jobs

Cloud Run jobs facilitate the efficient monitoring and debugging of PID image extraction. The extraction status of each image can be individually tracked on the Cloud Run jobs page, displaying whether it is pending, running, successful, or failed. Detailed logs for each image instance being processed are available which enables rapid identification and resolution of issues pertaining to specific images, thereby reducing resolution time and enhancing process efficiency.

10.3 Vertex AI Endpoints

Vertex AI provides tools to monitor model performance and resource utilization. The online prediction endpoints page tracks key metrics such as Model Requests per Second, Predictions per Second, Model Latency, and Prediction Error Percentage. Monitoring these metrics offers insights for scaling compute nodes for deployed models, and allows us to track performance and resource utilization.

11. Performance Testing

To validate the scalability and responsiveness of the Spyder Agent under various loads, performance testing was conducted using an internal simulation framework. These efforts focused on identifying bottlenecks and optimizing the system's ability to handle concurrent user requests with acceptable latency and error rates.

11.1 Testing Methodology and Tools

A custom-built Python simulator (`load_tester.py`), available at https://github.com/HON-CORP/Project-SPYDER-WS2-GCP/tree/stage/spyder/text_extraction/src/notebooks/performance_testing, was used to simulate concurrent users interacting with the `chat_v2` endpoint of the Spyder Agent. The simulator allowed for configurable numbers of concurrent users and test durations, measuring key performance indicators including:

- Average, Median, P90, and P99 Latency
- Requests per Second (RPS)
- Total Requests Sent, Successful Requests, and Failed Requests
- Error Rate

Tests were conducted across different configurations of the Token Exchange (TE) and Agent Runtime (AR) services, which are critical components deployed on Google Cloud Run. The largest test simulated 100 concurrent users for one hour.

11.2 Key Findings and Improvements

Initial tests revealed some performance issues under high load, with average latencies as high as ~160 seconds and high error rates under load. Through iterative testing and targeted optimizations, a substantial reduction in average latency was achieved, bringing it down to approximately 33.68 seconds for an equivalent 100-user test.

The main configuration and code improvements that led to these tangible reductions are summarized below:

- **Runtime Environment Optimizations:**

- **Gevent for I/O Bound Services:** The Token Exchange (TE) service, being I/O bound, was configured to run with `gevent` workers. This allowed for more efficient handling of concurrent connections, significantly improving its responsiveness.
 - **Gunicorn Worker and Thread Resizing for Agent Runtime (AR):** The Agent Runtime (AR) service's `gunicorn` configuration was adjusted to use 17 workers and 8 threads. This provided more processing capacity and enabled the Cloud Run instance to scale out more effectively by forcing it to scale more often.
 - **Increased vCPU and Minimum Instances:** Resources allocated to both TE and AR services were increased (e.g., TE: 4 GiB, 4 vCPU, Min Instance 2; AR: 8 GiB, 8 vCPU, Min Instance 2). This provided more computational power and reduced cold-start latencies.
 - **Adjusted Max Concurrency:** The maximum concurrency for the AR service was lowered to 30. This strategy, counter-intuitively, improved scaling behavior by forcing Cloud Run to provision new instances more aggressively as load increased, preventing individual instances from being overloaded.
- **Model and Timeout Adjustments:**
 - **Gemini Flash 2.5 Adoption:** Where applicable, the Gemini 2.5 Flash model was utilized for text extraction and generation tasks. As noted in Section 3.3, this model offers a faster response time, contributing to overall latency improvements.
 - **Explicit Gunicorn Timeouts:** Default timeouts at the `gunicorn` level (e.g., 100 seconds for TE, 90 seconds for AR) were set to kill hanging processes, preventing resource exhaustion and cascading failures.
- **Code-Level Enhancements:**
 - **Agent Initialization Code Reorganization:** The agent initialization code was refactored to reduce the number of clients and agent managers instantiated per request. This minimized internal overhead and improved processing efficiency.

These improvements directly address the "Sizing & Scaling" considerations outlined in Section 16.5, enabling the Agent Runtime Cloud Run instances to scale better and handle larger workloads more efficiently.

11.3 Detailed Test Results

Below are the results from two significant tests: an initial baseline (Test 1) demonstrating the system's performance before optimizations, and the optimal configuration achieved (Test 11). Intermediate tests (Test 2-10) showed a progressive improvement as optimizations were applied.

Test 1: Baseline Performance (100 concurrent users for 60s)

- **Configuration:**
 - TE: Standard gunicorn 1 worker, 8 threads, 4 GiB, 2 vCPU, Concurrency 80, Min instance 1
 - AR: Standard gunicorn 1 worker, 8 threads, 4 GiB, 2 vCPU, Concurrency 80, Min instance 1
- **Summary:** High latency and significant error rate due to Gateway Timeouts.
- **Metrics:**
 - Total test duration: 352.49 seconds
 - Total requests sent: 110
 - Successful requests: 80
 - Failed requests: 30
 - Error rate: 27.27% (primarily HTTP 504 Gateway Timeout)
 - Min latency: 26.5416 seconds
 - Max latency: 299.7252 seconds
 - Average latency: 160.0383 seconds
 - Median latency: 160.8868 seconds
 - Latency P90: 271.5189 seconds
 - Latency P99: 300.5216 seconds
 - Requests per second (RPS): 0.23

Test 11: Optimal Configuration Performance (100 concurrent users for 5m)

- **Configuration:**
 - TE: **gevent** workers=2, timeout=100, 4 GiB, 4 vCPU, Concurrency 100, Min instance 2
 - AR: Standard **gunicorn** 17 workers, 8 threads, timeout=90, 8 GiB, 8 vCPU, Concurrency 30, Min instance 2
 - Model used: Gemini Flash 2.5
- **Summary:** Significant reduction in latency and 0% error rate, demonstrating robust performance under load.
- **Metrics:**
 - Total test duration: 370.49 seconds
 - Total requests sent: 943
 - Successful requests: 943
 - Failed requests: 0
 - Error rate: 0.00%
 - Min latency: 16.3859 seconds
 - Max latency: 99.1969 seconds
 - Average latency: 33.6753 seconds
 - Median latency: 32.6138 seconds
 - Latency P90: 49.8110 seconds
 - Latency P99: 70.4899 seconds
 - Requests per second (RPS): 2.55

11.4 Conclusion and Further Considerations

The performance testing efforts successfully identified and mitigated key bottlenecks in the Spyder Agent services, leading to a substantial improvement in latency and stability under high concurrency. These improvements primarily focus on sizing and scalability of the Cloud Run services and agent logic, distinct from general provisioned throughput improvements for underlying LLMs (as discussed in Section 16.1).

Further optimizations and ongoing monitoring should continue to ensure the system meets evolving workload demands. Future work could include exploring further reduction of end-to-end latency by addressing the recommendation to "Remove the Token Exchange Service" (Section 16.2) if deemed necessary based on real-world usage patterns and expected workload, and continued tuning based on expected workload and usage as the project evolves.

12. Error Handling

A robust, multi-layered error handling strategy is implemented throughout the Spyder Project Assist system to ensure resilience, prevent catastrophic failures, and provide detailed diagnostics for debugging. The strategy differs based on the component's architecture—whether it is a long-running asynchronous pipeline or a real-time user-facing service—but generally combines explicit error trapping, stateful logging, and graceful failure notifications.

12.1. Ingestion Pipeline Error Handling

The Ingestion Pipeline is designed for high-throughput, asynchronous processing, and its error handling is architected to be resilient and granular, ensuring that the failure of a single document does not halt the entire batch.

1. **Workflow-Level Error Trapping:** The core of the strategy lies within the main `spyder-doc-ingestion-wf-v1.yaml` workflow. The processing of each individual file URI occurs within a `try/except` block inside the main parallel loop (`for singleUri in uriList`). This is a critical design choice, as it isolates failures. If converting a specific DOCX file fails or if Document AI cannot process a particular PDF, the `except` block catches the error. It then appends the failed URI and a detailed error message (containing the step name and error details) to the `failedUris` and `failedUrisDetailedLogs` workflow variables, respectively. The loop then continues to the next file, ensuring the batch can complete.
2. **Explicit Failure Logging:** At the conclusion of a workflow run, the `callWriteStatus` step invokes the `spyder-write-processing-status` Cloud Function. This function's primary role is to externalize the error state. It takes the `successfulUris`, `failedUris`, and `failedUrisDetailedLogs` lists from the workflow's state and writes them to distinct `.txt` files in a designated GCS bucket. This creates a persistent, auditable record of exactly which files succeeded and which failed, along with the specific reason for failure for each one. This is invaluable for support and for re-processing failed documents.
3. **Filtering Supported Failures for Retry:** In addition to logging all failed files, the system also creates a separate text file that contains only those failures corresponding to supported file types (such as `.docx` or `.pdf`). This filtered list is saved in the same GCS folder as the general failed files list. During retries, this supported-only list can be used as input instead of the full failure list, ensuring that only retry-eligible files are reprocessed — avoiding redundant failures from unsupported formats.

4. **Component-Level Timeouts and Retries:** At a lower level, individual calls within the workflow to Cloud Functions and Cloud Run jobs have implicit and sometimes explicit retry policies. HTTP-triggered functions will retry on specific HTTP error codes (e.g., 5xx). This provides a degree of transient error handling (e.g., a temporary network issue or a service being momentarily unavailable) without needing explicit logic in the workflow itself.

12.2. Text Extraction (Q&A) Error Handling

Error handling for the real-time Text Extraction service is focused on failing gracefully and providing immediate, informative feedback to the user.

1. **Tool-Level Input Validation:** The `document_search` tool in `agent_runtime/tools.py` begins by attempting to parse the input string and the `search_filter_json`. If the input is malformed, it will raise an exception that is caught, and the tool returns a user-friendly error message like `"[Error: Invalid input format...]"` back to the agent. This prevents the RAG chain from being invoked with invalid parameters.
2. **Graceful Failure in RAG Chain:** The core RAG logic within `create_multi_pdf_engine` is wrapped in standard Python error handling. If any part of the chain fails—for instance, if the Vector Search index is unreachable or the LLM API returns an error—the exception is caught. The `document_search` tool will then return a message indicating that an error occurred during processing, such as `"[Error: Could not retrieve an answer...]"` to the agent, which then relays this to the user.
3. **Handling of "No Answer":** A key part of the design is handling the scenario where the RAG system finds no relevant information. The system prompt for the Q&A LLM (`qa_system_prompt`) explicitly instructs the model: `If you don't know the answer, just say that you don't know`. This is a form of "soft" error handling that prevents the model from hallucinating or providing fabricated answers, instead correctly informing the user that the knowledge is not available in the document set.

12.3. Document Generation Error Handling

The Document Generation flow combines the synchronous error feedback of the agent tools with the robust, asynchronous error handling of the backend pipeline.

1. **Upfront Input Validation:** The `document_generation` tool in `agent_runtime/tools.py` performs critical upfront validation. Its most important check is: if not `search_filter_dict` or (`'opportunity_num'` not in `search_filter_dict` and `'project_num'` not in `search_filter_dict`):. If a project or opportunity number (which is mandatory for this flow) is missing, the tool immediately returns a clear error message "[Error: Project number or Opportunity number is required...]" to the user, and the backend workflow is never triggered. This prevents wasted processing cycles.
2. **Template Selection Failure:** The `select_template_controlled` utility has its own error handling. If it cannot confidently match the user's request to an available template in GCS (e.g., the fuzzy match score is too low and the LLM fallback fails), it returns `None`. The `document_generation` tool checks for this and returns an error message like "[Error: Could not identify the correct document template...]" if no template is found.
3. **Asynchronous Handler Errors:** Inside the `spyder-document-generation-handler`, the entire `document_generation_task_handler` is wrapped in a `try...except` block. If any part of the process fails (e.g., loading the template from GCS, a failure in the RAG engine during content generation, or an issue saving the final file), the exception is caught. The handler logs the detailed traceback to Cloud Logging for debugging and, crucially, sends an error notification email to the requesting user via the `send_failure_email` utility. This ensures the user is informed that their request failed and is not left waiting indefinitely for a document that will never arrive.

12.4. Image Digitization Error Handling

- **Robust Logging and Error Handling:** The image digitization process incorporates comprehensive logging and error-handling, utilizing the Python logging module for detailed logs and traces. This provides granular insight into each step, from image text extraction, entity detection and classification, line detection to final graph construction process, aiding in workflow monitoring and issue identification.
- **Exponential Backoff Strategy:** A key feature of the error handling is the implementation of an exponential backoff strategy, specifically for services prone to quota exhaustion errors in cloud environments such as `Document AI` and `multimodalembdding` model calls. This mechanism gradually increases retry delays after successive failures, preventing service overload and facilitating graceful recovery, thereby enhancing pipeline resilience.
- **Fault Isolation and Detailed Error Tracking:** The architecture is designed for fault isolation, with each image processed within its own dedicated Cloud Run job. This ensures that errors in one image's processing do not affect others. All errors encountered in individual Cloud Run jobs are meticulously tracked and accessible on the Cloud Run jobs execution page, offering a centralized and comprehensive overview of job statuses for efficient identification, diagnosis,

and resolution of processing failures, ultimately maintaining the integrity and throughput of the workflow.

13. Recommendations and Best Practices

13.1 Text Extraction

13.1.1. Recommended Modifications for Citations Improvement

Here are two recommended modifications to enhance citation accuracy:

1. **Stricter Relevance Score Filtering:** Currently, the code has a placeholder to filter citations based on a relevance score (`relevance_score > 0.80`). We will formally implement and fine-tune this threshold. By only citing sources with a high relevance score from the re-ranker, we can filter out documents that were retrieved but were likely not central to forming the final answer.
2. **LLM-Based Fact-Checking Step:** A more advanced approach would be to add a final step after the answer is generated. This step would involve another LLM call that is given the final answer and the content of the source chunks and is prompted to "double-check" and return only the IDs of the chunks that directly support statements made in the answer. This reduces the risk of "over-citing" and ensures every citation is directly traceable to the generated text. I will add this to the backlog for future implementation.

13.1.2. Implementing Hybrid Search

The Problem: Currently, our search system is very good at understanding the *meaning* behind a user's question (this is called semantic search). However, it can sometimes struggle when a user is looking for something with a very specific keyword, like a part number or an exact phrase ("`XYZ-123`"), because it focuses more on the context than the exact letters.

The Recommendation: We should implement **Hybrid Search**. This approach combines our current meaning-based search with a traditional **keyword search**. It's like getting the best of both worlds: the system can understand both what the user *means* and what they *literally* wrote.

How It Would Work:

1. When we ingest documents, we'll prepare them for both types of search.
2. When a user asks a question, the system will perform two searches simultaneously: one for meaning and one for exact keywords.
3. The system will then intelligently merge the results from both searches, ranking them to find the most relevant information.

Expected Impact: This will lead to a significant improvement in search quality. The system will be much better at handling all types of queries, resulting in more accurate context being sent to the AI. This means users will get more relevant and precise answers. Refer to [this notebook](#) for more information.

13.2 Document Generation and Ingestion pipeline

- **12.2.1. Specify Queries with Filters for Better Context:** The quality of a generated document is highly dependent on the relevance of the context retrieved by the RAG engine. Users should be encouraged to provide specific identifiers in their requests. A query like "Create a proposal for project USP-019208" is significantly more effective than "Create a proposal". The system's `get_search_filter` utility is designed to extract these identifiers to create a metadata filter, which dramatically narrows down the search space and provides the RAG engine with highly relevant context, leading to a more accurate and detailed final document.
- **12.2.2. Design Effective and Specific <prompt> Tags:** The content of the <prompt> tags within the .docx templates serves as the direct instruction to the RAG engine for each section. Vague prompts like <prompt>summary</prompt> will yield generic results. For superior output, prompts should be specific and instructive. For example, a better prompt would be <prompt>Provide a one-paragraph executive summary for the [Project Name] project, focusing on the key deliverables, timeline, and budget.</prompt>. This level of detail guides the LLM to generate more precise and useful content, directly improving the quality of the final generated document. The prompt extraction logic is handled in the `document_generation_task_handler`.
- **12.2.3. Additional Prompt Improvement Tips:**
 - **Explicitly specify the content scope:**
Use "current content" if you want information only from the **current index**, and use "historic content" if the answer should draw from **historic index**.

- **Split complex prompts:**
If the user's query contains multiple intents or is too complex for a single paragraph, **split it into multiple focused sub-prompts** for better retrieval and generation.
- **Be specific with entities and context:**
Clearly mention product names, timeframes, or document types (e.g., "Q4 2023 policy" or "Product ABC features") to guide retrieval toward the right chunk.
- **Use natural language, but avoid ambiguity:**
Write prompts as naturally as possible, but avoid vague terms like "that", "this", or "those" without clarifying what they refer to.
- **Include role or perspective if relevant:**
Add context like "As a sales manager..." or "From a legal compliance perspective..." if the answer should reflect a specific point of view.
- **Limit scope to what you need:**
Don't ask for "everything"—narrow down the query to what's truly needed to avoid overwhelming retrieval or off-topic results.
- **Anchor the prompt with known facts:**
If you already know part of the answer (e.g., "The policy was published in March 2024..."), include it to help focus the retrieval on related content.
- **Avoid compound questions when possible:**
Break compound queries like "What is the onboarding process and who approves expenses?" into two to improve precision.
- **Test prompts for retrieval quality:**
Periodically evaluate if the prompt brings the **most relevant chunks**. Tune structure and keywords if you observe irrelevant or missing data in responses.

13.3 Agent

New agent frameworks are constantly coming out; this is an area that is experiencing constant innovation. In the beginning of this project, we used Vertex AI Agent Engine's (formerly named Reasoning Engine) Langchain prebuilt agent implementation. We moved to using Langchain's ReAct agent as it provides more flexibility. Towards the end of the engagement, we recommended that the team move to Langchain's tool-calling agent implementation. This solution is more robust than a ReAct agent; ReAct agents rely on natural language to handle tool calls, which can break with new model releases (e.g. Gemini 2.5 pro seemed to have issues with parsing older versions of ReAct agents). Tool-calling agents are more efficient and handle tool calls through structured json. A more detailed listing of agent types is shown below:

Agent Type	Intended Model Type	Supports Chat History	Supports Multi-Input Tools	Supports Parallel Function Calling	Required Model Params	When to Use	API
Tool Calling	Chat	✓	✓	✓	tools	If you are using a tool-calling model	Ref
OpenAI Tools	Chat	✓	✓	✓	tools	[Legacy] If you are using a recent OpenAI model (1106 onwards). Generic Tool Calling agent recommended instead.	Ref
OpenAI Functions	Chat	✓	✓		functions	[Legacy] If you are using an OpenAI model, or an open-source model that has been finetuned for function calling and exposes the same functions parameters as OpenAI. Generic Tool Calling agent recommended instead	Ref
XML	LLM	✓				If you are using Anthropic models, or other models good at XML	Ref
Structured Chat	Chat	✓	✓			If you need to support tools with multiple inputs	Ref
JSON Chat	Chat	✓				If you are using a model good at JSON	Ref
ReAct	LLM	✓				If you are using a simple model	Ref
Self Ask With Search	LLM					If you are using a simple model and only have one search tool	Ref

Each of these agents takes a slightly different approach to answering questions, calling tools, and producing results back to the user. Below is an explanation of ReAct agents vs tool-calling agents.

1. [ReAct Agent](#):

ReAct is based on the 2022 paper (revised March 2023) titled [ReAct: Synergizing Reasoning and Acting in Large Language Models](#)

This paper was fundamental in the space of generative AI agents and introduced the idea of combining previous LLM concepts (Chain-of-Thought reasoning, Action and function invoking) together to produce an agent that is capable of breaking down complex tasks into subtasks, solving these and then combining results iteratively.

Below is an example from the ReAct paper that shows how using reasoning and actions iteratively can improve the quality of responses:

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod ✗

(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch ✗

(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control ...

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ...

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

Act 4: Finish[yes] ✗

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys] ✓

(2) AlfWorld

You are in the middle of a room. Looking quickly around you, you see a cabinet 6, a cabinet 1, a coffee machine 1, a countertop 3, a stove burner 1, and a toaster 1.
Your task is to: Put some pepper shaker on a drawer.

(2a) Act-Only

Act 1: Go to drawer 1
Obs 1: The drawer 1 is closed.

Act 2: Open drawer 1
Obs 2: You open the drawer 1. The drawer 1 is open. In it, you see a dish sponge 2, and a spoon 1.

Act 3: Go to sinkbasin 1
Obs 3: On the sinkbasin 1, you see a dish sponge 3, a spatula 1, and a spoon 2.

Act 4: Take peppershaker 1 from sinkbasin 1
Obs 4: Nothing happens.

Act 5: Take peppershaker 1 from sinkbasin 1
Obs 5: Nothing happens. ✗

(2b) ReAct (Reason + Act)

Act 1: Think[First I need to find a pepper shaker...more likely to appear in cabinets (1-6), countertops (1-3), ...]
Act 2: Go to cabinet 1
Obs 2: On the cabinet 1, you see a vase 2.
 (Here the agent go to cabinet 1, then cabinet 2, 3, then countertop 1 and 2)

Act 6: Go to countertop 3
Obs 6: On the countertop 3, you see a apple 1, a bread 1, a pepper shaker 1, and a vase 3.

Act 7: Take pepper shaker 1 from countertop 3
Obs 7: You pick up the pepper shaker 1 from the countertop 3.

Act 8: Think[Now I find a pepper shaker 1. Next, I need to put it in/on drawer 1.]
Act 9: Go to drawer 1
Obs 9: Drawer 1 is closed.

Act 10: Open drawer 1
Obs 10: You open Drawer 1 ...

Act 11: Put pepper shaker 1 in/on drawer 1
Obs 11: You put pepper shaker 1 in/on the drawer 1. ✓

2. Tool-Calling Agent

Most leading LLMs (e.g. Gemini, Claude, GPT, etc) support some form of function calling; that is to say, they've been pre-trained to parse, invoke, and understand the response of functions in code. Since the mechanisms of doing function calling are all different across the leading LLMs on the market, Langchain has an agent type that provides a single interface for function calling within an agent.

77

The Tool-Calling agent implementation for Spyder uses the [create_tooling_calling_agent](#) Langchain function, which takes in an LLM, a set of tools, and a prompt:

```
Python
base_llm = ChatVertexAI(
    model_name='gemini-1.5-pro-002',
    temperature=0,
    top_p=1.0,
    top_k=1,
    safety_settings={
        HarmCategory.HARM_CATEGORY_HATE_SPEECH: HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
    },
    convert_system_message_to_human=True
)

prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a helpful assistant. Answer all questions to the best of your ability. "
        ),
        ("placeholder", "{chat_history}"),
        ("human", "{input}"),
        ("placeholder", "{agent_scratchpad}"),
    ]
)

agent_runnable = create_tool_calling_agent(base_llm, tools, prompt)
```

13.4 P&ID Image Extraction

13.4.1 Data Augmentation

To enhance the robustness and generalization capabilities of the model for P&ID image extraction, it is crucial to implement data augmentation techniques to artificially inflate the size and diversity of the training dataset, thereby exposing the model to a wider range of variations that it might encounter in real-world scenarios. This expanded and diversified dataset will, in turn, lead to a model that is more resilient to unseen variations, less prone to overfitting, and ultimately, exhibits superior generalization capabilities when deployed in a production environment.

13.4.2 Human-in-the-loop validation and iterative model improvement

Implement a human-in-the-loop validation process for P&ID data extraction. This step will enable manual review and correction of low-confidence extractions, and the corrected data can be used to retrain the models. By establishing a continuous feedback loop, human-validated corrections will continuously improve the performance of P&ID image extraction models. To enhance the vessel detection model, it can be trained on compressed images smaller than 1.5 MB. The vessel dataset can be boosted, particularly for less frequent vessel types like horizontal vessels. Augmentation techniques, including creating patches of varying sizes and resolutions and adding rotations, can improve the vessel detection dataset and, consequently, the model's performance.

14. Code Repository

This section provides an overview of the source code repository for the Spyder Project Assist system, including its structure, branching strategy, and code quality standards.

14.1. Repository Details

The complete source code for this project is hosted on the corporate GitHub Enterprise instance within the HON-CORP organization.

- **Repository Name:** [Project-SPYDER-WS2-GCP](#)
- **Visibility:** The repository is **Private**, accessible only to authorized Honeywell personnel.
- **Purpose:** This repository serves as the single source of truth for all application code, service configurations, workflow definitions, and documentation for the Spyder Google Webservice (GCP) project.

14.2. Branching Strategy

The repository follows a Gitflow-inspired branching strategy to manage development, testing, and production releases in a structured and predictable manner.

- **main:** This is the primary branch representing the stable, production-ready codebase. Merges into `main` are controlled and typically come from the `stage` branch after successful testing.
- **stage:** This is the main integration branch for pre-production. All feature branches are merged into `stage` for end-to-end testing in a dedicated staging environment.
- **Feature Branches:** Individual development for new features or bug fixes is done on short-lived feature branches. These are created from `main` or `stage` and merged into `stage` upon completion via pull requests, triggering automated checks and deployment pipelines.

14.3. Directory Structure

The repository is organized into high-level directories for different capabilities (e.g., text extraction, image digitization) and standard software engineering practices. It includes:

- **Top-Level Directories:**

- `spyder/`: The primary directory containing all application source code, organized by capability.
- `tests/`: Houses all unit, integration, and end-to-end tests for the various system components.
- `.gitignore`: Standard Git configuration file to exclude build artifacts, local environment files, and dependencies from version control.
- `pylintrc`: A configuration file for the Pylint linter, used to enforce a consistent code style and identify potential errors across the Python codebase.
- `README.md`: The main documentation file containing the project overview, setup instructions, and deployment details.

- **Capability-Specific Directories (within `spyder/`):**

- `text_extraction/`: This directory contains all code related to the Text Extraction (Q&A) and Document Generation capabilities. The code analyzed for this TDD resides here. Its structure is further detailed below.
- `image_digitisation/`: This directory is designated to house the source code for the Image Digitization process.

- **`spyder/text_extraction/src/` Directory Structure:**

This directory contains the source code for the serverless functions, services, and workflows that power the Text Extraction and Document Generation features.

- `agent_runtime/`: Contains the user-facing Agent Runtime service. This includes the Flask application (`main.py`), the agent logic (`agent.py`), the definition of the agent's tools (`tools.py`), and related utilities.
- `cloud_functions/`: A monorepo-style directory containing the source code for all individual Cloud Functions, with each function isolated in its own subdirectory (e.g., `spyder-check-new-files-v1/`, `spyder-document-generation-handler/`).
- `cloud_run/`: Holds the source code for containerized services. This includes the `docx-to-pdf-job-v3/` service, which contains the Python code and Dockerfile for the LibreOffice file converter.
- `workflows/`: This directory houses the YAML definitions for all Cloud Workflows, such as `spyder-doc-ingestion-wf-v1.yaml` and `spyder-doc-gen-workflow.yaml`.
- `notebooks/`: Contains Jupyter notebooks used for experimentation, data analysis, and developing proofs-of-concept.
- `tasks_queue/`: Likely contains configurations or scripts related to task queuing mechanisms used by the system.

- `requirements.txt` & `setup.py`: Standard Python files for managing package dependencies and project setup.
- **spyder/image_extraction/src/ Directory Structure:**

This directory contains the code for the PID digitization process and for updating files related to entity classification, which is essential for solution improvement.

Core Modules & Processes:

- `main.py`: The primary entry point that initiates the image extraction process.
- `entity_detection` module: Utilizes an object detection model hosted on a Vertex AI online prediction endpoint to identify entities within images.
- `line_detection` and `segmented_line_detection` modules: Employ the Line Segment Detector (LSD) to perform line detection and recognize segmented lines.
- `ocr_extraction` module: Dedicated to extracting text from PID images.
- `graph_construction` module: Builds a graph based on the entity and line detections from preceding modules.
- `entity_classification` and `update_embeddings` modules: These modules are responsible for classifying detected entities (symbols, vessels, and line types) into their respective categories. They also update the pickle files to enhance the classification accuracy.

Supporting Components:

- `utils` module: Contains common utility functions and business logic necessary for mapping the output to Honeywell's specific format.
- Pickle files (`.pkl`): Store embeddings for symbols, vessels, and line types, which are crucial for the entity classification process.

Deployment & Automation:

- `build_image` script: A script used to build a Docker image of the PID digitization solution and push it to the GCP Artifact Registry (`image-extraction-repository`).
- `Dockerfile`: Defines the configuration for creating the Docker image that runs as a Cloud Run job.
- `image-extraction-workflow.yaml`: Houses the YAML configuration required to create the `image-extraction-workflow` on Cloud Workflows.

Notebooks:

- Notebooks for experimentation, data exploration, object detection model training and evaluation, inference, entity classification, and line detection are stored in `spyder/image_extraction/notebooks/`.

15. Honeywell Security Requirements

15.1 Data Encryption

- Data at rest is encrypted using AES-256 by default, without any action required from the user. This includes multiple layers of encryption at the hardware, infrastructure, and storage device levels.
- When a user sends a request to a Google Cloud service, data is secured in transit; providing authentication, integrity, and encryption, using HTTPS with a certificate from a web (public) certificate authority. Any data the user sends to the GFE is encrypted in transit with Transport Layer Security (TLS). GFE negotiates a particular encryption protocol with the client depending on what the client is able to support. GFE negotiates more modern encryption protocols when possible. API interactions with Google Cloud over TLS.

15.2 Secure Network Infrastructure

- Dedicated, isolated network segment for AI operations.
- Network segmentation and access control to restrict unauthorized access.
- AI component deployed as API endpoint with access only with Service Account. The API is backed by Cloud Run which is used to run fully managed Serverless microservices.

15.3 Access Control:(Authentication Authorization)

- Multi-factor authentication (MFA) for system access.
- Role-based access control (RBAC) to limit access based on user roles.
- Regular review and auditing of access privileges.

15.4 Firewalls and Intrusion Prevention Systems:

- Advanced firewall solutions to monitor and control incoming and outgoing network traffic. The UI end point is an internal website with no access to external traffic.
- Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) for real-time threat detection and response.

15.5 Anti-Malware and Anti-Virus Solutions/ DDoS Protection:

- Regularly updated anti-malware and antivirus software.
- Automated scanning of files and communications for malicious content.
- Implement solutions to detect and mitigate Distributed Denial of Service (DDoS) attacks for the UI end point which is an internal website.
- Serverless components will handle traffic spikes during an attack.

15.6 API Security:

- API endpoint can be connected with authentication and rate limiting.
- Regularly scan and test APIs for vulnerabilities.

15.7 Logging and Incident Response:

- Comprehensive logging of all network activities.
- Establish an incident response plan for quick action in case of a security breach.

15.8 Data Privacy:

- There are explicit instructions within the RAG prompt that specify to avoid using sensitive information that could be found within the context of the retrieved document chunks:

None

```
"4. Security and Privacy:"  
" 4.1. Absolutely refuse to answer any questions related to passwords,  
credentials, personal identifiable information, or other sensitive security data. A  
noteable exception to this rule is if the word 'Confidential' is found in the context,  
that is okay you can answer the question then."  
"\n\n"
```

15.9 Secure Development:

- Conduct regular security code reviews and vulnerability assessments.
- Follow secure coding practices to prevent common vulnerabilities (e.g., SQL injection, cross-site scripting).

15.10 Regular Updates and Patch Management:

- Keep all software components, libraries, and frameworks up to date with security patches.
- Establish a process for timely patch management to address vulnerabilities.

15.11 Data Backup and Recovery:

- Implement regular data backups and test the restore process to ensure data integrity and availability.
- BigQuery is a fully managed enterprise Data Warehouse built to store and analyze massive datasets. It provides a way to ingest, store, read, and query megabytes to petabytes of data with consistent performance without having to manage any of the underlying infrastructure. BigQuery offers an industry leading 99.99% uptime SLA. This is made possible by BigQuery's regional architecture that writes data in two different zones and provisions redundant compute capacity. Because BigQuery is a regional service, it is the responsibility of BigQuery to automatically handle the loss of a machine or even an entire zone. The fact that BigQuery is built on top of zones is abstracted from users.

15.12 User Training and Awareness:

- Provide security training and awareness programs for all personnel involved with the AI solution.
- Educate users on recognizing and reporting security threats.

15.13 Data Retention Policies:

- Define clear data retention and data disposal policies to minimize the risk of data exposure.
- Automatically delete unnecessary data according to Honeywell policies.

15.14 Secure Deployment Environment:

- Secure the infrastructure where the AI solution is deployed, including firewalls, intrusion detection systems, and network security.

15.15 Regular Security Testing:

- Conduct penetration testing and vulnerability scanning regularly to identify and address security weaknesses.

15.16 Secure User Interfaces:

- There is no User Interface provided as part of this deliverable. There is a parallel development effort by the Honeywell team to build the User Interface.

15.17 Documentation and Training Materials:

- This document along with the recordings of the knowledge sessions can be used as training materials for personnel involved in the project using this document.

15.18 Vendor Security Assessment:

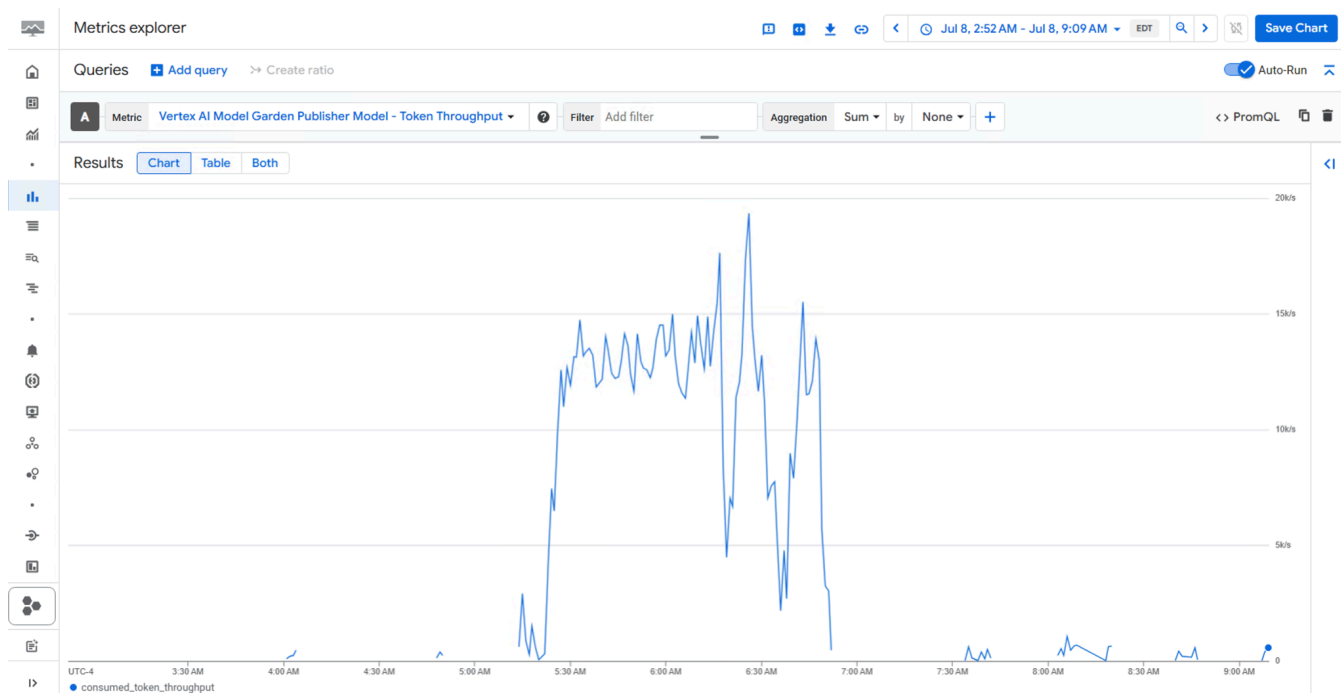
- This solution does not include any 3rd party vendors..

16. Future Scalability Considerations

The following are recommended updates to the Knowledge Assist system to involve the latency and throughput of the system.

1. **Provisioned Throughput:** The current system and its workloads are likely not going to experience many 429 errors, but as the system and the number of users grow this will become a bigger challenge. To request a raise in PT, please find the step by step guide [here](#). Based on the provision throughput observed in last most as well as in the load testing <refer the attached graphs>, the initial suggested PT values were:

10k tokens/sec in UAT and 20k tokens/sec in prod environment - Gemini 2.5 Pro
15k tokens/sec in UAT and 30k tokens/sec in prod environment - Gemini 2.5 Flash



2. **Remove the Token Exchange Service:** We are currently using an additional Cloud Run service to do authentication of the end user. This additional hop adds in significant end-to-end latency, contributing an additional second or two each time. The better alternative is to replace this service with either:
 - a. Deploy the frontend service in GCP (recommended), and generate GCP identity tokens that can be sent directly to the backend agent runtime service.

- b. Use Workload Identity Federation (WIF) to impersonate service account identities on the frontend service and use these identities to produce GCP identity tokens that can be sent directly over to the backend agent runtime service, rather than needing to go through the token exchange service first.
 - c. Use service account keys to impersonate service account identities on the frontend service, which can also be used to produce GCP identity tokens. Note: neither of these options will necessarily do Authorization of the end user; this addressed authentication instead
- 3. **Create Dedicated Frontend Service:** Today, the Knowledge Assist Agent is attached to the Maintenance Assist Agent has a single-use tool (e.g. for specific tenants, the Azure-deployed MA agent has been given 1 tool to use, and this is the Knowledge Assist Agent, so it effectively just forwards the request to the GCP based solution). The MA system was chosen because of its availability in terms of a pre-deployed, production-ready UX. This decision has incurred significant technical debt in terms of latency and throughput, and likely adds an additional 4-5 second latency with each request. The currently frontend service is deployed as a React Application in Azure, which communicates with the MA Azure Agent, which then communicates with the GCP-deployed token exchange service, which then communicates with the Backend Agent Runtime service, and then all the way back. There are too many links, and too many cloud environments. The recommendation is shown below:

- a. Current State:

React Application (Azure) <-> MA Agent (Azure) <-> Token Exchange Svc (GCP) <-> Agent Runtime (GCP)

- b. Recommended Future State:

React Application (GCP) <-> Agent Runtime (GCP)

- 4. **Use Flash Models where applicable:** The current agent runtime uses Gemini-2.5-pro for nearly all inferencing; pro models can be slower than flash models. It is recommended to use a flash model for the agent base model, and pro models for RAG and NL2SQL applications/tools.
- 5. **Sizing & Scaling:** The Cloud Run instances (agent runtime and token exchange service) have a number of configurable options. They should be set up to handle the approach workload. Options include:
 - a. Memory and CPU

- b. Request timeout
 - c. Maximum concurrent requests per instance
 - d. Minimum number of instances
 - e. Maximum number of instances
6. **Metadata dropdowns:** The current system uses a series of LLM calls to pull out key metadata (e.g. project numbers, opportunity IDs, etc.) This process adds several additional LLM calls just to extract this information from the original query. These queries can be replaced by adding in dropdowns in the UI for key metadata fields, which would get rid of the need to use LLMs to extract this information. There is also the added benefit of providing end users with a structured, limited option for metadata fields that are available rather than having end users guess which projects/opportunities are available and have been indexed by the system.
7. **Model upgrades:** For future model upgrades, the recommendation is to use the steps as outlined in the standard upgrade playbooks.

<https://cloud.google.com/vertex-ai/generative-ai/docs/migrate-to-v2>

17. Cost Estimation

17.1. Cost Estimation for RAG Document Ingestion Pipeline (1,000 Documents)

This document outlines the estimated one-time cost to ingest and process 1,000 PDF documents in the **us-central1** region. The costs are calculated based on official Google Cloud pricing as of July 2025 and a set of core assumptions about the data volume and processing requirements.

17.1.1. Core Assumptions

- **Total Documents:** 1,000 PDF files.
- **Average Pages per Document:** 50 pages.
- **Total Pages to Process:** 50,000 pages.
- **Chosen Document Parser:** Document AI OCR Parser at **\$1.50 per 1,000 pages**.
- **Embedding Model:** Vertex AI Gemini Embedding at **\$0.15 per 1 million input tokens**.
- **Total Tokens for Embedding:** 31.25 million tokens.
- **Cloud Region:** **us-central1** (Tier 1 pricing).

17.1.2. One-Time Ingestion Cost Breakdown

The **us-central1** region benefits from Tier 1 pricing for infrastructure services like Cloud Run, resulting in a lower overall cost compared to other regions.

Service	Calculation Details (us-central1)	Estimated Cost
Document AI	Processing 50,000 pages with the Layout Parser at \$10.00 per 1,000 pages.	\$75.00
Vertex AI Embeddings	Generating embeddings for 31.25 million tokens with the Gemini model at \$0.15 per 1M tokens.	\$4.69
Cloud Run	Compute for processing jobs (~30k vCPU-seconds and ~60k GiB-seconds) in a Tier 1 region.	\$1.01
Vertex AI Vector Search	One-time cost to build the index from ~0.2 GiB of vector data at \$3.00/GiB.	\$0.57
Google Cloud Storage	Storing 10 GB of source PDFs for one month in us-central1, including operations.	\$0.21
Cloud Workflows	Orchestrating 1,000 pipeline executions.	\$0.08

17.1.3. Total Estimated Ingestion Cost

- **Calculated Raw Total: ~\$81.56**

It is best practice to include a buffer to account for variations in document complexity, processing retries, and other unforeseen factors.

- **Recommended Budget (with 15% Buffer): ~\$94.00**

17.1.4. Estimated Ongoing Operational Costs (Post-Ingestion in us-central1)

Ongoing costs for hosting the infrastructure are also lower in **us-central1**.

- **Vertex AI Vector Search Hosting:**
 - *Estimate: ~\$68 - \$75 per month* (for a single **e2-standard-2** node in **us-central1**).
- **Google Cloud Storage:**
 - *Estimate: ~\$0.20 per month* for storing 10 GB.
 -

17.1.5. Disclaimer

This is a planning estimate. Actual costs may vary based on the final implementation, specific document characteristics (especially page count), and any changes in Google Cloud pricing. We recommend setting up billing alerts to monitor costs closely during and after deployment.

17.2. Cost Estimation for Document Generation Pipeline

17.2.1. Core Assumptions

Below estimate is based on the following operational assumptions for your RAG pipeline:

- **Total Prompts per Document:** 158 (150 standard prompts + 8 from the original query including parsing filters, template etc).
- **Average Input Tokens per LLM Call:** **6,050 tokens**. This assumes each of the 3 retrievers returns ~2,000 tokens of context, plus the original prompt.
- **Average Output Tokens per LLM Call:** **500 tokens** on average for each generated response.
- **Cloud Run Execution:** The function takes an average of **8 seconds** to process each prompt (including retrieval and generation) and is configured with **2 vCPUs** and **8 GB of RAM** in **us-central1**.
- **LLM Pricing:** "Gemini 2.5 Pro" rates of **\$1.25 per 1M input tokens** and **\$10.00 per 1M output tokens**.

17.2.2. Cost Breakdown per Document

Here is a detailed breakdown of the costs involved in generating one complete document.

Service	Calculation Details	Estimated Cost

 Vertex AI (Gemini 2.5 Pro)	Input: (158 prompts × 6,050 tokens) = 955,900 tokens = \$1.20 Output: (158 prompts × 500 tokens) = 79,000 tokens = \$0.79	\$1.99
 Cloud Run	Total execution of ~1,264 seconds on a 2 vCPU, 8 GB instance.	\$0.09
Total	Combined cost for one document	~\$2.08

17.2.3. Other Negligible Cost Factors

It's important to remember that this calculation covers the *marginal cost* of generating one document. Your total cloud bill will also include fixed, ongoing costs for the underlying infrastructure that are not part of this per-document price:

- **Cloud Logging:** Costs associated with logging the executions of your Cloud Run functions.
- **GCP Workflows:** Cost associated with GCP workflows service

17.3 Cost Estimation for P&ID Digitization

17.3.1 Core Assumptions

The following assumptions underpin the cost estimations for P&ID digitization pipeline:

- **Total P&ID Drawings:** 1,000 drawings are assumed for digitization.
- **Average Patches per Drawing:** Each drawing is divided into approximately 30 patches for processing.
- **Average Symbols per Image:** Each image contains an average of 100 symbols, vessels, or special line types.
- **Chosen Document Parser:** Document AI OCR Parser (Document OCR parser) is used, priced at **\$1.5 per 1,000 pages**.
- **Average Model Uptime:** Model endpoints are assumed to be online for 12 hours.
- **Average Processing Time:** Each image digitization is estimated to take 30 minutes.

- **Embedding Model:** Vertex AI multimodal embedding model is used, priced at **\$0.0001 per image input**.

17.3.2 One-Time Setup and Initial Processing Cost Breakdown

Service	Calculation Details	Estimated Cost
Document AI	Processing 1,000 images with 30 patches each, at a rate of \$1.5 per 1,000 pages.	\$45.00
AutoML Model Prediction Endpoint	Object detection from images using an AutoML trained model at \$2.002 per node hour, with the node kept live for 12 hours.	\$24.02
Multimodal Embedding Calls	Classification of detected objects using multimodal embedding model calls (1,000 images with 100 objects per image).	\$10.00
Cloud Run Jobs	Compute for processing jobs (8 vCPU, 20 minutes per image, 1,000 images, 32 GiB memory).	\$249.60
Google Cloud Storage	Storing 10 GB of P&ID images (1,000 images at ~1 MB each) for one month in us-central1, including operations.	\$0.20
Cloud Workflows	Orchestration of 1,000 pipeline executions, with 3 steps per execution.	\$0.03

AutoML Model Training	Non-recurring AutoML object detection model training (assuming 50 hours of training at \$3.465 per node hour).	\$173.25
------------------------------	--	-----------------

- **Estimated running cost:** ~\$329
- **Estimated one-time cost for updating models:** ~\$173

17.3.3 Disclaimer

This is a planning estimate. Actual costs will vary significantly based on the number and complexity of P&ID drawings, the specificity of required custom AutoML models, and any changes in Google Cloud pricing. We strongly recommend setting up billing alerts to monitor costs closely during and after deployment.

18. Glossary of Acronyms

- ANN: Approximate Nearest Neighbor
- API: Application Programming Interface
- CI/CD: Continuous Integration/Continuous Delivery
- GCP: Google Cloud Platform
- GCS: Google Cloud Storage
- IAM: Identity and Access Management
- IDS: Intrusion Detection Systems
- IPS: Intrusion Prevention Systems
- JSON: JavaScript Object Notation
- JWT: JSON Web Token
- LLM: Large Language Model
- LSD: Line Segment Detector
- MFA: Multi-factor Authentication
- NLU: Natural Language Understanding
- OCR: Optical Character Recognition
- P&ID/PID: Piping and Instrumentation Diagram
- PSO: Professional Services Organization
- PT: Provisioned Throughput
- RAG: Retrieval-Augmented Generation
- RBAC: Role-based Access Control
- RPS: Requests Per Second
- UAT: User Acceptance Testing
- VPC: Virtual Private Cloud

- WIF: Workload Identity Federation
- PMT: Performance Material and Technology
- HBT: Honeywell Building Technologies
- SPS: Safety and Production Solutions
- SBU: Strategic Business Unit
- Mechanical Systems & Components
- UOP: Universal Oil Products
- HPS: Honeywell Process Solutions
- GBE: Gold Business Enterprise
- BA: Building Automation
- IAM: Identity Access Management
- HMI: Human-Machine Interface
- MA: Maintenance Assist